

Growth & Campaign Intelligence

ImpactIQ — AI-Powered Campaign Uplift & Incremental Impact Engine

৬. ট্র্যাক ০৪: Growth & Campaign Intelligence

এমন AI সিস্টেম তৈরি করা, যা একটি MFS (Mobile Financial Service)-কে আরও কার্যকরভাবে গ্রো করতে সাহায্য করবে। AI বিশ্লেষণ করে বের করবে—

- কোন customer-কে টার্গেট করা উচিত
- কোন merchant-কে যুক্ত করা উচিত
- কোন offer দেওয়া উচিত
- কোন সময়ে offer/campaign দেওয়া উচিত
- এবং কোন intervention দিলে সবচেয়ে বেশি অতিরিক্ত (incremental) value তৈরি হবে।

Project Opportunities — সম্ভাব্য প্রজেক্ট

১. Next-Best-Offer Engine

প্রতিটি customer-এর জন্য কোন offer সবচেয়ে বেশি relevant, AI সেটার ভিত্তিতে offer-গুলোকে ranking করবে।

২. Campaign Response Prediction

কোন customer একটি নির্দিষ্ট campaign-এ response দেওয়ার সম্ভাবনা বেশি, AI সেটা পূর্বাভাস দেবে।

৩. Uplift Modeling

কোন customer-এর আচরণ একটি নির্দিষ্ট offer/intervention দেওয়ার কারণে পরিবর্তিত হওয়ার সম্ভাবনা বেশি, AI সেটা শনাক্ত করবে।

৪. Campaign Budget Optimizer

একটি নির্দিষ্ট budget বিভিন্ন customer group বা offer-এর মধ্যে কীভাবে ভাগ করলে সবচেয়ে বেশি incremental outcome পাওয়া যাবে, AI সেটা নির্ধারণ করবে।

৫. Lifecycle Orchestration

Customer-এর lifecycle-এর কোন পর্যায়ে সে আছে—

Acquisition → Activation → Retention → Win-back

তার ভিত্তিতে পরবর্তী কোন engagement/action নেওয়া উচিত, AI সেটা recommend করবে।

৬. Offer Fatigue Detection

একই ধরনের offer বা campaign বারবার দেওয়ার ফলে customer-এর interest কমে যাচ্ছে বা বিরক্তি তৈরি হচ্ছে কিনা, AI সেটা শনাক্ত করবে।

৭. Experiment Intelligence

বিভিন্ন campaign variant-এর ফলাফল তুলনা করে AI বলবে কোন ধরনের experiment পরবর্তীতে test করা যেতে পারে।

#এই customer এমনিতেই transaction করবে” — এটা জানা তুলনামূলকভাবে দুর্বল insight।

IMPACTIQ- এই customer-কে offer দেওয়ার কারণেই সে transaction করেছে” — এটা অনেক বেশি গুরুত্বপূর্ণ insight।

SOLUTION

Customer Data → AI Prediction → Next Best Offer → Uplift Score → Campaign Selection → Budget Optimization → Experiment Feedback

Key:

Topic:

Problemটা আসলে কী

Real-world MFS use case

AI/ML কীভাবে solve করবে

কী ধরনের data লাগবে

কোন ML technique/model ব্যবহার করা যায়

Input → AI → Output flow

কীভাবে evaluate করবে

কীভাবে incremental impact measure করবে

Implementation difficulty

Hackathon/demo-তে কীভাবে দেখানো যায়

কোন কোন topic একসাথে combine করা যায়

১. Next-Best-Offer Engine

প্রতিটি customer-এর জন্য কোন offer সবচেয়ে বেশি relevant, AI সেটার ভিত্তিতে offer-গুলোকে ranking করবে।

১১. Hackathon/Demo-তে কীভাবে দেখানো যায়?

এখানে তোমরা খুব সুন্দর একটা dashboard বানাতে পারো।

Screen 1 — Customer Search

Customer ID: 10245

Recent activity:

Recharge: 8
Merchant Pay: 2
Send Money: 4
Avg Txn: ₹245
Last Txn: 1 day ago

Screen 2 — AI Recommendation

NEXT BEST OFFER

 Recharge Cashback

AI Score 87%
Response Probability 82%
Estimated Uplift 31%

Why?

- ✓ Frequent recharge user
- ✓ Responded to similar offers
- ✓ Active recently
- ✓ Preferred time: 8 PM

Screen 3 — Alternatives

#1 Recharge Cashback 87%
#2 Bill Payment Cashback 71%
#3 Merchant Cashback 43%
#4 Send Money Discount 29%

Screen 4 — Business Impact

Target Customers: 100,000

AI Selected: 25,000

Expected conversions:

Without AI: 3,200

With AI: 4,100

Estimated incremental:

+900

আর সবচেয়ে সুন্দর demo হবে যদি “**Why this offer?**” explanation দেখাও।

কারণ financial domain-এ শুধু prediction নয়, recommendation-এর rationale-ও গুরুত্বপূর্ণ।
Real-world banking NBO architectures-এ eligibility, propensity, timing এবং recommendation rationale আলাদা করে বিবেচনা করা হয়। Bank.AI

১২. কোন কোন topic-এর সাথে combine করা যায়?

এখানেই ৭টা topic-এর connection বোঝা শুরু হবে।

NBO + Campaign Response Prediction

সবচেয়ে natural combination।

Customer



Response Prediction



Offer Ranking



Next Best Offer

NBO + Uplift Modeling ★

এটা আরও advanced।

Response probability

+

No-offer probability



Incremental uplift



Best offer

এতে তোমরা শুধু “কে respond করবে” না, বরং “কার behaviour offer-এর কারণে বদলাবে” সেটা target করতে পারবে।

NBO + Offer Fatigue Detection

ধরো AI দেখল:

Customer received:

Recharge Offer × 7

Response:

5 → 3 → 2 → 1 → 0

তখন system বুঝতে পারে:

এই customer-এর জন্য একই ধরনের offer-এর effectiveness কমছে।

তখন ranking-এ সেই offer-এর score কমিয়ে দিতে পারে।

NBO + Lifecycle Orchestration

Customer যদি:

New user



Activated



Regular user



Declining



Dormant

তাহলে প্রতিটি stage-এ NBO আলাদা হতে পারে।

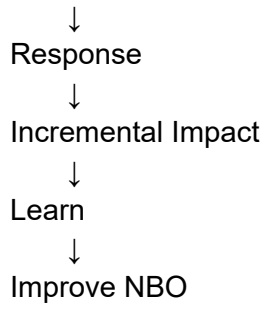
NBO + Experiment Intelligence

AI recommendation চালানোর পাশাপাশি:

Offer A vs Offer B



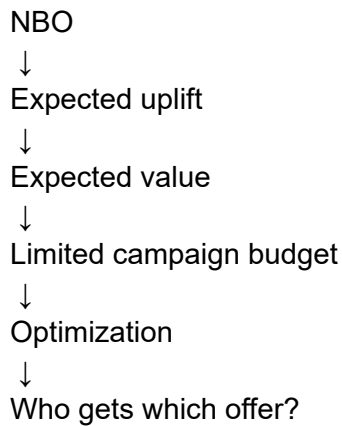
Experiment



এতে system একটা **learning loop** হয়ে যায়।

NBO + Campaign Budget Optimizer

এটা সবচেয়ে বড় architecture-এর দিকে যায়:



Budget-constrained incentive recommendation-কে optimization problem হিসেবেও formulate করা যায়; recent research-এ uplift-based incentive recommendation এবং budget allocation একসাথে বিবেচনা করা হয়েছে। [DOI](#)

সবচেয়ে গুরুত্বপূর্ণভাবে NBO-টা এভাবে মনে রাখো

Basic NBO:

Who wants what?

Better NBO:

Who is likely to respond to which offer?

Advanced NBO:

Who will actually change their behaviour because of which offer?

Mature NBO:

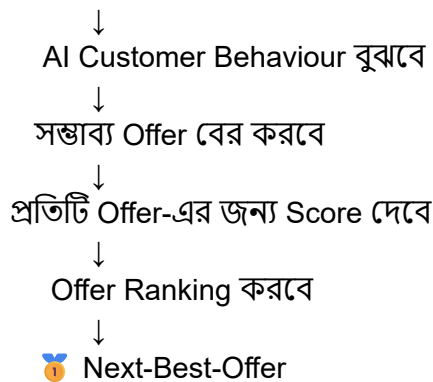
Which customer should receive which offer, at what time, through which channel, considering eligibility, expected incremental impact, business value, budget and offer fatigue?

AI term:

ধরো MFS-এর কাছে ১০ লাখ customer এবং ২০টা offer আছে। AI-এর কাজ হলো প্রতিটি customer-এর জন্য কোন offer সবচেয়ে উপযুক্ত সেটা বের করা।

AI-এর কাজের পুরো flow

Customer-এর পুরোনো Data



একটা example দেখি

ধরো **Rahim** নামের একজন customer:

- গত মাসে ১০ বার mobile recharge করেছে
- average recharge = ২৫০ টাকা
- merchant payment খুব কম
- আগেও recharge cashback নিয়েছে
- গত ৩ দিন ধরে active
- সাধারণত রাত ৮টার দিকে transaction করে

তার জন্য ৪টা offer আছে:

Offer	AI Score
📱 Recharge Cashback	0.89
🛒 Merchant Cashback	0.42
💡 Bill Payment Cashback	0.35
💰 Send Money Discount	0.21

AI তখন বুঝতে পারে:

Rahim-এর historical behaviour অনুযায়ী **Recharge Cashback** সবচেয়ে relevant।

তাই:

Next Best Offer → Recharge Cashback

কিন্তু AI এই 0.89 score পেল কীভাবে?

এখানেই Machine Learning।

আগের অনেক customer-এর historical data AI-কে দেওয়া হবে।

ধরো:

Customer data

+

Offer data

+

আগের campaign

+

Customer response

↓

Machine Learning Model

↓

Probability / Score

আগের data থেকে model pattern শিখবে।

যেমন সে শিখতে পারে:

যারা নিয়মিত recharge করে + আগে recharge cashback নিয়েছে + সম্প্রতি active হয়েছে → তাদের recharge offer নেওয়ার সম্ভাবনা বেশি।

এটা মানুষ manually rule লিখে দিচ্ছে না; historical data থেকে ML model relationship শিখছে।

তাহলে AI কী predict করবে?

Basic version-এ:

P(Customer will respond to Offer)

যেমন:

Rahim + Recharge Cashback



0.89

Rahim + Merchant Cashback



0.42

অর্থাৎ AI বলছে:

“এই customer-এর জন্য কোন offer-এ response করার সম্ভাবনা বেশি?”

কিন্তু তোমাদের Track-এর Advanced অংশ আরও interesting

Brief-এ বলেছে:

“This customer will transact” is weaker than “this customer is likely to transact because of the offer.”

মানে AI শুধু response prediction করলে পুরো advanced solution হবে না।

ধরো:

Customer A

Offer না দিলেও:

Transaction probability = 90%

Offer দিলে:

Transaction probability = 92%

AI বলছে:

Incremental impact $\approx +2\%$

অন্যদিকে Customer B:

Without offer = 20%

With offer = 70%

তাহলে:

Incremental impact $\approx +50\%$

এখানে B-এর ক্ষেত্রে offer-এর actual impact বেশি।

এই অংশে AI/ML-এর uplift modeling / causal inference ব্যবহার করা যায়।

তাই Advanced NBO হবে

AI একসাথে দুইটা জিনিস বুঝবে:

1. Response prediction

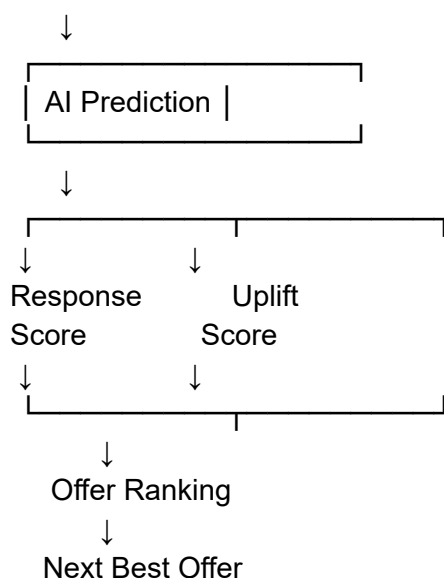
“এই customer offer নিলে transaction করবে?”

2. Uplift prediction

“Offer দেওয়ার কারণেই কি customer-এর behaviour পরিবর্তন হবে?”

তারপর:

Customer Data



আর AI শুধু Offer নির্বাচন করলেই শেষ না

একটা mature system চাইলে AI আরও বলতে পারে:

কাকে?

Customer 1024

কী দিতে হবে?

Recharge Cashback

কখন দিতে হবে?

8 PM

কেন দিতে হবে?

Frequent recharge + previous cashback response

কতটা impact আশা করা যায়?

Estimated uplift: 31%

কাকে offer না দিলেও হবে?

যাদের organic transaction probability already খুব high।

এক লাইনে বললে:

AI historical customer behaviour এবং previous campaign results দেখে প্রতিটি customer-এর জন্য বিভিন্ন offer-এর সম্ভাব্য response/impact predict করবে, তারপর সেই score অনুযায়ী সবচেয়ে উপযুক্ত offer-টি rank করে Next-Best-Offer হিসেবে recommend করবে।

Campaign Response Prediction — Full Project Plan

1. 🎯 আমরা আসলে কোন Problem Solve করছি?

একটি MFS যখন campaign চালায়, তখন সাধারণত অনেক customer-কে একই campaign পাঠানো হয়।

ধরো:

“Mobile Recharge করলে ২০ টাকা Cashback”

MFS-এর ১০ লাখ eligible customer আছে।

কিন্তু—

- সবাই recharge করবে না
- সবাই cashback-এ interested না

- কেউ এমনিতেই recharge করত
- কেউ campaign দেখেও ignore করবে
- কেউ campaign পাওয়ার পর প্রথমবার recharge করবে

তাই মূল problem:

কোন customer-কে campaign দিলে সে response করার সম্ভাবনা বেশি—এবং campaign পাঠিয়ে কোথায় সত্যিকারের অতিরিক্ত value তৈরি হবে?

আমাদের system এই decision-টা data-driven করবে।

2. ❌ Traditional System-এর Problem

ধরো বর্তমানে campaign team একটা rule ব্যবহার করে:

যারা গত 30 দিনে ≥ 5 বার recharge করেছে

↓

তাদের সবাইকে campaign পাঠাও

এতে সমস্যা:

Problem 1 — Over-targeting

যে customer এমনিতেই recharge করত তাকেও offer দেওয়া হচ্ছে।

Problem 2 — Under-targeting

শুধু simple rule-এর কারণে এমন customer বাদ পড়ে যেতে পারে, যে campaign পেলে response করত।

Problem 3 — Campaign Cost

সবাইকে SMS/push/in-app campaign পাঠালে unnecessary cost তৈরি হয়।

Problem 4 — Customer Fatigue

একই ধরনের campaign বারবার পেলে customer ignore করতে শুরু করতে পারে।

Problem 5 — Correlation $\neq$ Incremental Impact

যে customer campaign পাওয়ার পর transaction করেছে, সে হয়তো campaign ছাড়াও transaction করত।

এই শেষ সমস্যাটাই সবচেয়ে গুরুত্বপূর্ণ।

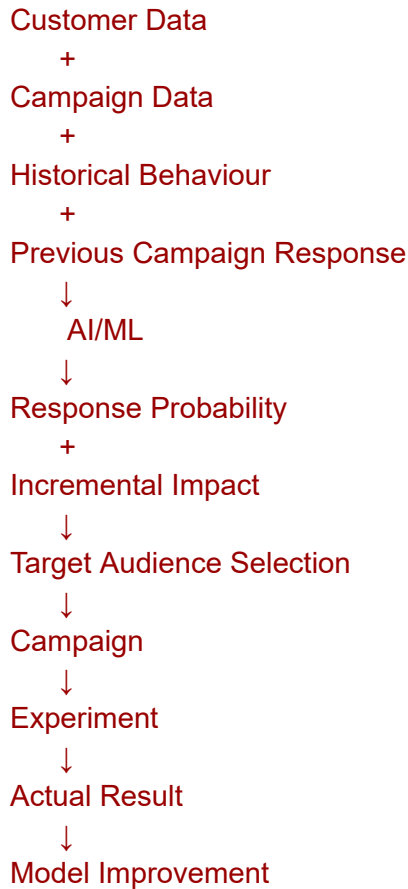
3. 💡 আমাদের Proposed Solution

আমরা একটি:

AI-powered Campaign Response & Incremental Impact Prediction System

তৈরি করব।

System-এর কাজ:



4. 🤖 AI-এর Proper Use কোথায়?

AI-কে শুধু একটি fancy prediction feature হিসেবে ব্যবহার করব না।

AI-এর মূল কাজ হবে:

① Customer behaviour শেখা

Historical data থেকে pattern বের করবে।

② Response probability predict করা

এই customer campaign-এ response করবে কতটা?

③ Complex relationships detect করা

যেমন:

Recharge frequency

+

Recent activity

+

Previous cashback response

+

Average transaction

+

Time

+

Campaign type

একসাথে response-এর ওপর কী প্রভাব ফেলছে সেটা model শিখবে।

④ Customer ranking

সবচেয়ে likely responders-দের ranking করবে।

⑤ Advanced uplift estimation

শুধু response না—

Campaign-এর কারণে response করার সম্ভাবনা কতটা বেড়েছে?

এটাও estimate করবে।

5. Complete AI Logic

System-এ কয়েকটি layer থাকবে।

Layer 1 — Eligibility Engine

এখানে AI লাগবে না।

Rule-based filtering হবে।

উদাহরণ:

Campaign:
Recharge Cashback

Eligibility:
✓ Active account
✓ Eligible customer segment
✓ Minimum recharge requirement
✓ Offer validity
✓ Not already redeemed

ফলে:

1,000,000 Customers



Eligibility Rules



650,000 Eligible

Layer 2 — Customer Behaviour Engine

প্রতিটি customer-এর behaviour থেকে features তৈরি হবে।

যেমন:

Transaction Frequency
Recharge Frequency
Average Transaction
Days Since Last Transaction
Merchant Usage
Send Money Usage
Bill Payment Usage
Preferred Transaction Time
Recent Activity

Layer 3 — Campaign Understanding

Campaign-এর নিজস্ব features থাকবে।

যেমন:

Campaign Type
Offer Type
Cashback Amount
Minimum Transaction

Validity
Channel
Campaign Duration
Target Product

Layer 4 — Historical Response

এটিই AI-এর learning material।

ধরো customer আগে ১০টা campaign পেয়েছে:

Campaign	Response
Recharge Cashback	✓
Bill Cashback	✗
Merchant Cashback	✓
Recharge Discount	✓
Send Money Offer	✗

AI বুঝতে পারবে:

Customer কোন ধরনের campaign-এর প্রতি বেশি responsive।

Layer 5 — AI Response Prediction

এখন model বলবে:

এই customer + এই campaign → response probability কত?

উদাহরণ:

Customer C1024

Campaign: Recharge Cashback

Response Probability = 87%

আরেকজন:

Customer C8271

Campaign: Recharge Cashback

Response Probability = 19%

6. কোন ML Model?

আমরা model development-এ staged approach রাখব।

Baseline

Logistic Regression

কারণ:

- সহজ
- explainable
- probability দেয়
- benchmark হিসেবে useful

Main model

XGBoost / LightGBM

কারণ customer behaviour-এর nonlinear relationship ধরতে পারে।

Model comparison

আমরা compare করব:

Logistic Regression

vs

XGBoost

vs

LightGBM

তারপর validation data দিয়ে performance compare করব।

7. ★ Uplift / Incremental Impact Layer

এটাই system-এর advanced অংশ।

Response model বলছে:

“C1024 response করবে 87%।”

কিন্তু প্রশ্ন:

“Campaign না দিলে কী হতো?”

তাই আমরা দুইটা সম্ভাবনা estimate করতে চাই।

With Campaign

$P(\text{Response} \mid \text{Campaign})$

Without Campaign

$P(\text{Response} \mid \text{No Campaign})$

তারপর:

Uplift =

$P(\text{Response} \mid \text{Campaign})$

-

$P(\text{Response} \mid \text{No Campaign})$

উদাহরণ:

Customer A

With campaign = 90%

Without campaign = 87%

Uplift = +3%

Customer B

With campaign = 70%

Without campaign = 20%

Uplift = +50%

Customer A-এর response probability বেশি হলেও **Customer B campaign-এর জন্য বেশি incremental value তৈরি করতে পারে।**

এটাই system-এর advanced intelligence।

8. Targeting Logic

এখন শুধু response probability দেখে target করব না।

আমরা consider করব:

Response Probability
+
Estimated Uplift
+
Expected Transaction Value
+
Campaign Cost
-
Fatigue Risk

এর মাধ্যমে একটা **Campaign Priority Score** তৈরি করা যেতে পারে।

Conceptually:

Campaign Priority
=
Expected Incremental Value
-
Campaign Cost
-
Fatigue Penalty

Exact formula পরে experimentation-এর মাধ্যমে নির্ধারণ করব।

9. Offer/Campaign Fatigue Logic

এটা system-কে আরও realistic করবে।

ধরো:

Customer received
Recharge Campaign × 8

Response:

45%
38%
27%
19%
11%
6%
3%

1%

System detect করবে:

Repeated exposure → declining response

তখন:

Fatigue Risk = HIGH

এবং campaign priority কমে যাবে।

10. Timing Intelligence

একই customer সবসময় একই সময় active না।

ধরো:

Customer's transaction history

10 AM → Low

2 PM → Medium

8 PM → High

তাহলে system বলতে পারে:

Recommended campaign time: 8 PM

তবে timing prediction-কে প্রথম version-এ core model-এর সঙ্গে জটিলভাবে না জুড়ে আলাদা feature হিসেবে রাখা ভালো।

11. Campaign Targeting Output

শেষে campaign manager দেখতে পারবে:

Campaign:

20ট Recharge Cashback

Eligible Customers:

650,000

AI Recommended:

180,000

Breakdown:

High Priority	80,000
Medium Priority	60,000
Low Priority	40,000
Do Not Target	470,000

12. Explainable AI

AI শুধু বলবে না:

“C1024 = 87%”

বরং:

Why?

Customer C1024

- ✓ 8 recharge in last 30 days
- ✓ Active 1 day ago
- ✓ 62% previous cashback response
- ✓ Average recharge matches campaign
- ✓ Similar users responded positively

আর যদি uplift কম হয়:

⚠ High organic transaction probability

Customer likely to transact
even without campaign.

Estimated uplift: +3%

এটা খুব powerful insight।

13. Experiment Layer

Prediction-এর পরে actual campaign চালাতে হবে।

সব high-score customer-কে campaign পাঠিয়ে দিলে আমরা causal impact বুঝতে পারব না।

তাই:

Treatment Group

AI-selected customers-এর একটি অংশ campaign পাবে।

Control Group

Similar customers campaign পাবে না।

উদাহরণ:

Treatment = 20,000

Control = 20,000

তারপর compare:

Treatment conversion = 18%

Control conversion = 12%

Estimated incremental lift:

+6 percentage points

14. Feedback Loop

Campaign শেষ হওয়ার পর actual result আবার system-এ যাবে।

Prediction



Campaign



Actual Response



Treatment vs Control



Incremental Impact



New Training Data



Model Retraining



Better Prediction

এটা system-কে static ML model না রেখে **learning system** বানাবে।

15. 📊 কীভাবে Success Measure করব?

দুই ধরনের metrics থাকবে।

ML Metrics

- ROC-AUC
- PR-AUC
- Log Loss
- Precision@K
- Recall@K

Business Metrics

সবচেয়ে গুরুত্বপূর্ণ:

- Response Rate
 - Conversion Rate
 - Incremental Lift
 - Incremental Transactions
 - Revenue
 - Campaign Cost
 - ROI
 - Cost per Incremental Conversion
-

16. Hackathon Demo

Dashboard-টা এমন হতে পারে:

Campaign Selection

Campaign	
🎁 20ট Recharge Cashback	
Eligible: 650,000	

AI Analysis

AI TARGETING

High Response Probability
180,000

Estimated High-Uplift
92,000

High Fatigue Risk
18,000

Customer View

Customer: C1024

Response Probability 87%
Estimated Uplift 31%
Fatigue Risk LOW

Recommended Action:
TARGET

Another Customer

Customer: C7812

Response Probability 91%
Estimated Uplift 3%
Fatigue Risk LOW

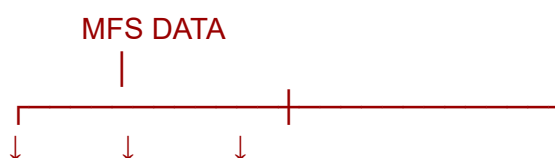
Insight:
Likely to transact even
without campaign.

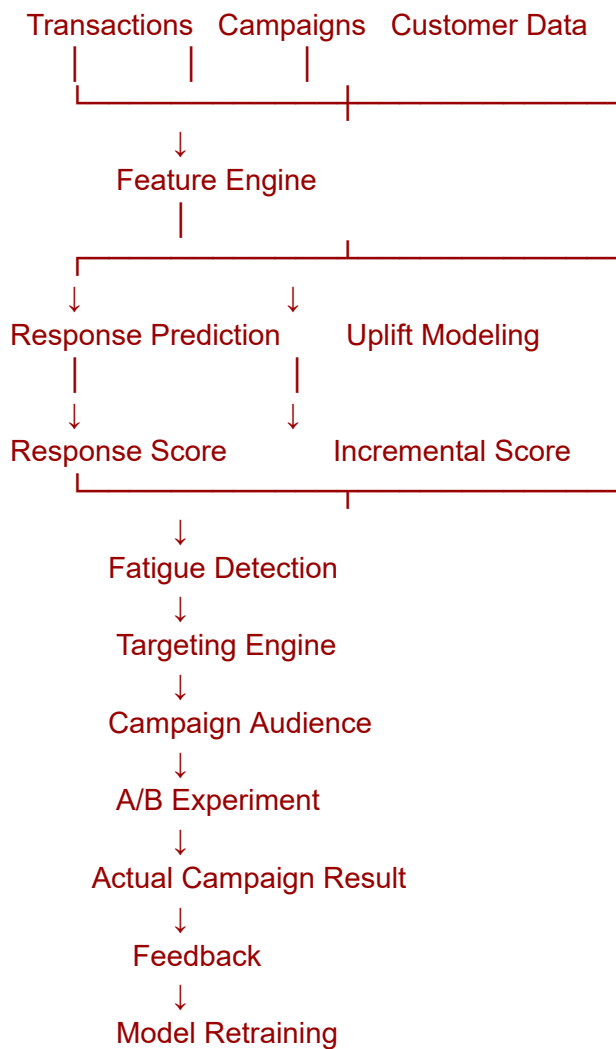
Recommended Action:
LOW PRIORITY

এই দ্বিতীয় example-টাই judge-কে দেখাবে যে তোমাদের system শুধু propensity model না।

17. System Architecture

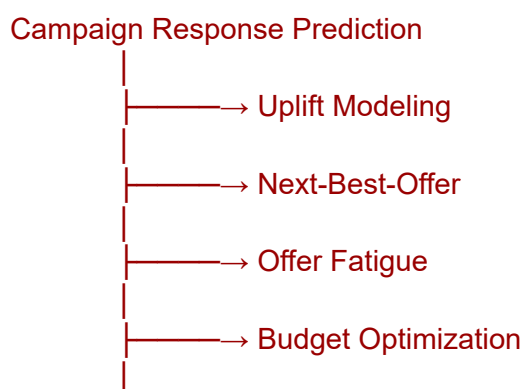
শেষে architecture দাঁড়াবে:

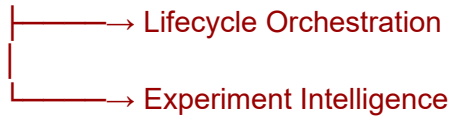




18. Track 04-এর অন্য Topic-এর সঙ্গে Connection

এই project একা কাজ করতে পারে, কিন্তু এটাকে পরে পুরো **Growth & Campaign Intelligence Platform**-এর অংশ বানানো যায়।





বিশেষ করে:

Response Prediction + Uplift

কে response করবে?



কার response campaign-এর কারণে হবে?



কাকে target করলে incremental value সবচেয়ে বেশি?

এটাই আমাদের core intelligence।



Final Problem Statement

তোমাদের project-এর problem statement আমি এভাবে define করতাম:

MFS platforms often send campaigns to large customer segments without accurately estimating individual customer response or distinguishing organic behaviour from campaign-driven behaviour. This can lead to inefficient targeting, unnecessary campaign costs, customer fatigue, and limited incremental value.

আমাদের solution:

An AI-powered campaign intelligence system that predicts individual customer response, estimates incremental campaign impact, detects campaign fatigue, and helps select the customers most likely to generate additional value from an intervention.

সহজ বাংলায়:

“সবাইকে campaign না দিয়ে, কাকে campaign দিলে সত্যিকার অর্থে অতিরিক্ত transaction/value তৈরি হবে—AI দিয়ে সেটা শনাক্ত করা।”



MASTER PLAN — AI-Powered Uplift & Incremental Campaign Intelligence

Core Idea

“কাকে offer দিলে তার behavior সত্যিই offer-এর কারণে পরিবর্তিত হবে?”

সিস্টেমের AI historical customer + campaign + treatment/control data থেকে শিখবে এবং প্রতিটি customer-এর জন্য Offer দিলে কী হতে পারে বনাম Offer না দিলে কী হতে পারে—এই দুইটা outcome estimate করবে।

1. Problem Definition

MFS platforms বড় customer base-কে campaign/offer দেয়। কিন্তু সব customer-এর ক্ষেত্রে offer দেওয়ার প্রয়োজন নেই।

কিছু customer:

- offer ছাড়াই transaction করত
- offer পেলেও transaction করত না
- offer পাওয়ার কারণে transaction শুরু/বাড়াত
- অতিরিক্ত campaign পেলে বিরক্ত/কম responsive হতে পারে

তাই মূল সমস্যা:

কোন customer-এর ক্ষেত্রে campaign-এর কারণে সত্যিকার incremental behavior/value তৈরি হবে তা আগে থেকে নির্ধারণ করা কঠিন।

ফলে হতে পারে:

- campaign budget waste
 - unnecessary cashback
 - low incremental transaction
 - unnecessary customer targeting
 - offer fatigue
 - organic transactions-এর জন্যও incentive দেওয়া
-

2. Proposed Solution

আমরা তৈরি করব:

AI-Powered Uplift & Incremental Campaign Intelligence Engine

এটি প্রতিটি eligible customer-এর জন্য estimate করবে:

$P(\text{Response} \mid \text{Offer})$

এবং

$P(\text{Response} \mid \text{No Offer})$

তারপর:

Individual Uplift

=

$P(\text{Response} \mid \text{Offer})$

-

$P(\text{Response} \mid \text{No Offer})$

এর ভিত্তিতে system identify করবে:

কাদের target করলে campaign-এর incremental impact সবচেয়ে বেশি হওয়ার সম্ভাবনা আছে।

3. 🧠 AI-এর Exact কাজ

এটা সবচেয়ে গুরুত্বপূর্ণ।

AI-এর কাজ শুধু response prediction না।

AI historical data থেকে customer behavior এবং campaign outcome-এর relationship শিখবে।

Input

Customer Behaviour

+

Campaign / Offer

+

Historical Treatment-Control Data

AI

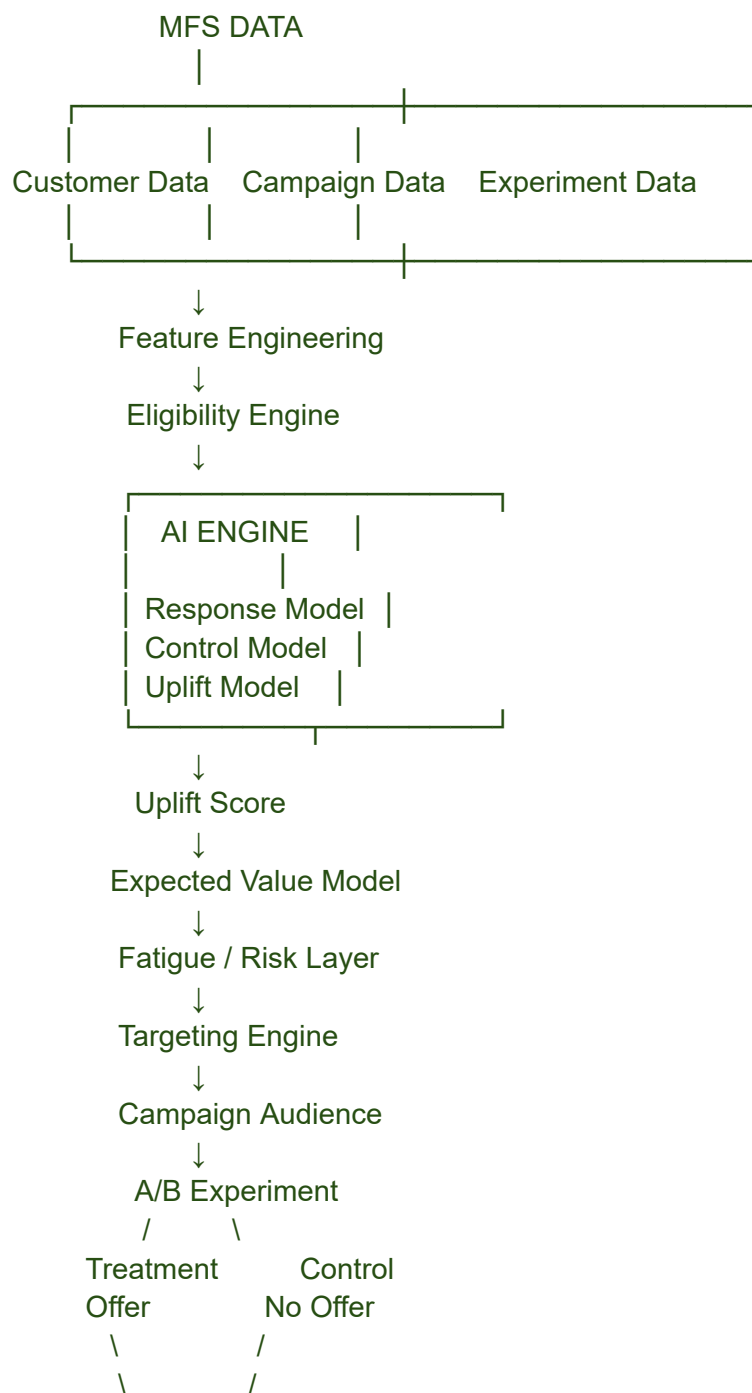
ML / Uplift Models

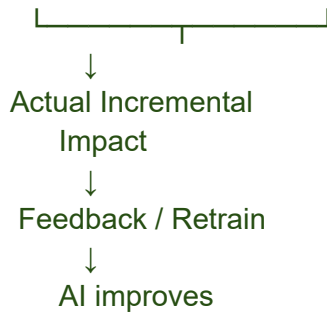
Output

Response if Offered

- + Response if Not Offered
 - + Uplift
 - + Expected Incremental Value
-

4. Complete System Architecture





5. Data Layer

AI ভালোভাবে কাজ করতে historical data দরকার।

Customer Features

customer_id
account_age
transaction_frequency
average_transaction_amount
recharge_frequency
merchant_payment_frequency
bill_payment_frequency
send_money_frequency
days_since_last_transaction
preferred_transaction_time

Campaign Features

campaign_id
offer_type
cashback_amount
minimum_transaction
campaign_duration
channel
target_product

Historical Campaign Features

campaign_received
campaign_opened
campaign_clicked
offer_redeemed
transaction_after_campaign
previous_campaign_count
days_since_previous_campaign

Experiment Data

সবচেয়ে গুরুত্বপূর্ণ:

customer_id
treatment/control
campaign_id
campaign_received
response
transaction_amount

কারণ Uplift Modeling-এর জন্য আমাদের জানতে হবে:

Offer পেয়েছিল কী না এবং তারপর কী হয়েছিল।

6. 🖌️ Feature Engineering

Raw data সরাসরি AI-তে যাবে না।

System customer behavior থেকে meaningful features তৈরি করবে।

উদাহরণ:

Recency

days_since_last_transaction

Frequency

transactions_last_7_days
transactions_last_30_days

Monetary

avg_transaction_amount
total_transaction_amount_30d

Campaign behaviour

campaigns_received_30d
campaigns_responded_30d
campaign_response_rate

Offer fatigue

days_since_last_offer
offers_received_7d
offers_received_30d

7. 🤖 AI Layer — মূল অংশ

এখানে system দুইটা outcome estimate করবে।

Model A — Treatment Model

Question:

Offer দিলে customer respond করার probability কত?

Customer Features

+

Offer Features



ML Model



$P(\text{Response} \mid \text{Treatment})$

Example:

C1024 → 78%

Model B — Control Model

Question:

Offer না দিলেও customer respond করার probability কত?

Customer Features



ML Model



$P(\text{Response} \mid \text{Control})$

Example:

C1024 → 24%

8. 🎯 Uplift Calculation

এখন:

Uplift

=
Treatment Probability
-
Control Probability

Example:

78% - 24%
= +54%

System output:

C1024 Estimated Uplift = +54%

অর্থাৎ campaign intervention-এর কারণে response বাড়ার estimated potential অনেক বেশি।

9. Four Customer Segments

Uplift model থেকে customer-দের behaviour অনুযায়ী চারটি group করা যায়।

Segment	Offer	No Offer	Meaning
Persuadable	Yes	No	 Offer-এর কারণে change
Sure Thing	Yes	Yes	এমনিতেই করবে
Lost Cause	No	No	Offer দিয়েও করবে না
Do Not Disturb	No	Yes	Offer দিলে potentially negative effect

Primary target

Persuadable customers

কারণ campaign-এর incremental value এখানেই বেশি হওয়ার সম্ভাবনা।

10. Expected Incremental Value

শুধু uplift দিয়ে final decision না নেওয়াই ভালো।

ধরো:

Customer A
Uplift = 50%
Expected transaction = 100ট

অন্যদিকে:

Customer B
Uplift = 30%
Expected transaction = 5,000ট

তাই business value-ও বিবেচনা করতে হবে।

Concept:

Expected Incremental Value
=
Uplift
×
Expected Transaction Value

তারপর campaign cost বিবেচনা করা যায়।

Expected Net Value
=
Expected Incremental Value
-
Campaign Cost

এতে AI output business decision-এর সাথে যুক্ত হবে।

11. 🧠 Offer Fatigue Layer

একজন customer-কে বারবার offer দেওয়া হলে response কমতে পারে।

System track করবে:

Offers received recently
+
Days since last offer
+
Previous offer responses
+
Campaign frequency

তারপর:

Fatigue Risk
Low / Medium / High

Final targeting-এ fatigue penalty ব্যবহার করা যাবে।

12. Targeting Engine

এখন system সিদ্ধান্ত নেবে:

Uplift
+
Expected Value
+
Fatigue
+
Eligibility

এর ভিত্তিতে:

Action 1

TARGET

High incremental potential.

Action 2

HOLD

Potential আছে, কিন্তু timing/fatigue/etc. বিবেচনা করা দরকার।

Action 3

DO NOT TARGET

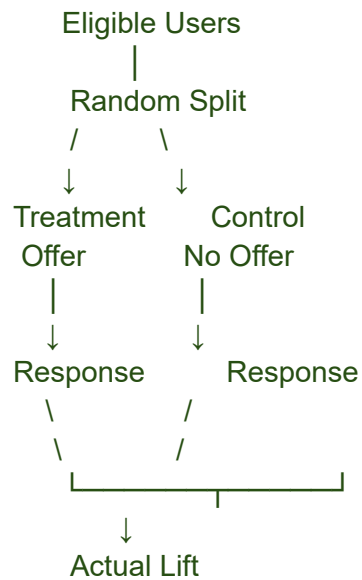
Customer এমনিতেই transaction করবে অথবা offer থেকে expected incremental benefit খুব কম।

13. Experiment Intelligence

এটা project-এর অত্যন্ত গুরুত্বপূর্ণ অংশ।

AI prediction করার পর campaign automatically blindly send করা যাবে না।

Eligible audience-এর মধ্যে:



Example:

Treatment conversion = 18%

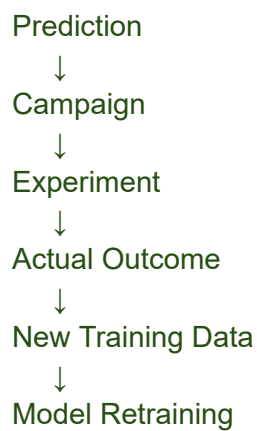
Control conversion = 11%

Observed Incremental Lift
= +7 percentage points

এটা AI-এর prediction-এর সাথে compare করা হবে।

14. Continuous Learning

System একবার model বানিয়ে থেমে থাকবে না।



↓
Better Prediction

অর্থাৎ system সময়ের সাথে campaign behaviour শিখবে।

15. Evaluation

শুধু ML accuracy দেখালে হবে না।

ML Metrics

- ROC-AUC
- PR-AUC
- Log Loss
- Precision@K
- Uplift/Qini-related evaluation where appropriate

Business Metrics

সবচেয়ে গুরুত্বপূর্ণ:

- Incremental conversion
 - Incremental transactions
 - Incremental revenue/value
 - Campaign cost
 - Cost per incremental transaction
 - ROI
 - Offer redemption
 - Customer fatigue
-

16. Demo Dashboard

Hackathon presentation-এর জন্য dashboard হবে:

Campaign Selection

Campaign:

[50ট Recharge Cashback ▼]

Eligible Customers:

1,250,000

AI Recommended:
185,000

Estimated Incremental Transactions:
31,400

তারপর:

Customer	P(Offer)	P(No Offer)	Uplift	Fatigue	Action
C001	94%	91%	+3%	High	Don't Target
C002	82%	25%	+57%	Low	Target
C003	75%	72%	+3%	Low	Hold
C004	65%	15%	+50%	Low	Target

17. 🔍 Explainable AI

Dashboard-এ শুধু:

Uplift = 57%

দেখালে user বুঝবে না কেন।

তাই explanation থাকবে:

Why this customer?

- ✓ High recharge frequency
- ✓ Responded to similar offers before
- ✓ Low recent campaign exposure
- ✓ Long time since last recharge
- ✓ High predicted incremental response

এতে business user AI-এর recommendation বুঝতে পারবে।

18. 🛠️ Recommended Tech Stack

Hackathon-এর জন্য unnecessarily complicated করার দরকার নেই।

Backend

Python
FastAPI
Pandas
Scikit-learn
XGBoost / LightGBM

Database

PostgreSQL / SQLite

Prototype হলে SQLite যথেষ্ট।

Frontend

React

Visualization

Recharts / Plotly

Model

প্রথম version:

Logistic Regression

তারপর:

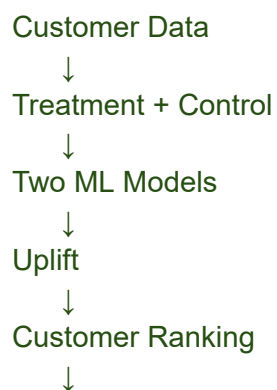
XGBoost / LightGBM

তারপর:

T-Learner / Two-Model Uplift

19. MVP বনাম Advanced Version

MVP



Dashboard

এতেই core idea প্রমাণ করা যাবে।

Advanced

এর সাথে:

- + Expected Value
 - + Offer Fatigue
 - + Explainability
 - + Campaign Optimization
 - + Experiment Tracking
 - + Automated Retraining
-

20. 🏆 Final Product Concept

শেষ পর্যন্ত তোমাদের system এমন হবে:

Campaign Manager:

“আমার ৫০৬ recharge cashback campaign আছে। কাদের দেব?”

System বলবে:

AI analysis complete.

Eligible: 1,250,000
Recommended: 185,000

High Uplift Users: 142,000
Medium Uplift: 43,000
Avoid / Low Value: 1,065,000

Expected Incremental Transactions:
31,400

Expected Incremental Value:
৳X

Estimated Campaign Cost:
৳Y

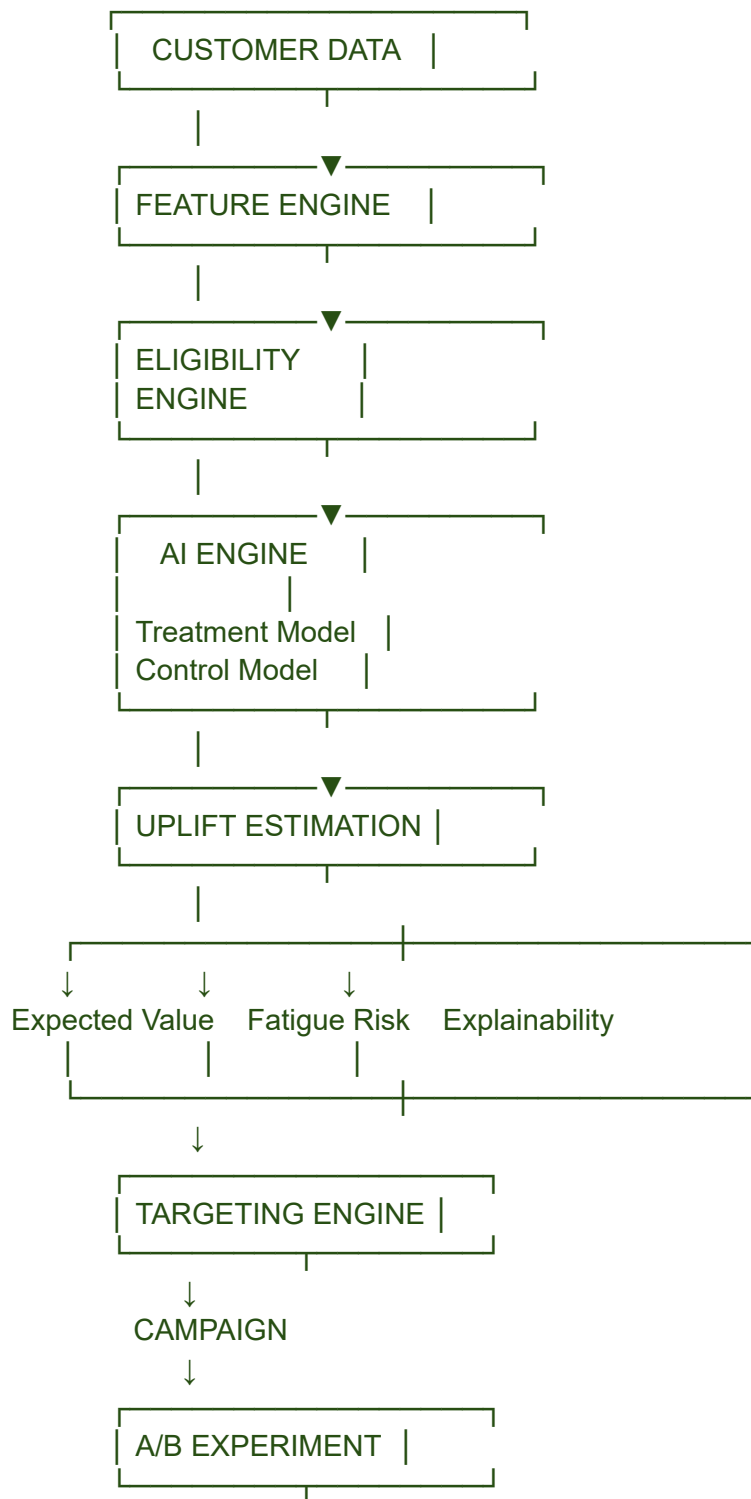
এবং customer-level explanation:

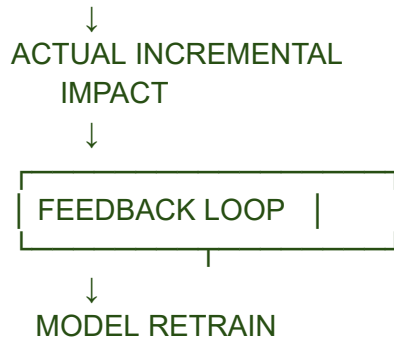
C002 — Target

Response with offer: 82%

Response without offer: 25%
Estimated uplift: **+57%**
Fatigue: Low

Final Master Flow





One-line pitch

“Instead of predicting who will respond to an offer, our AI predicts who will respond because of the offer—and prioritizes customers based on their expected incremental value.”

এটাই আমি তোমাদের Uplift Modeling-এর core master solution হিসেবে ধরতাম। সবচেয়ে গুরুত্বপূর্ণ হলো, AI-এর জায়গাটা পরিষ্কার: AI historical treatment/control data থেকে individual counterfactual outcome estimate করবে; business rules শুধু eligibility/constraints handle করবে।

8. Campaign Budget Optimizer — আসল ধারণা

মূল প্রশ্ন:

“আমার budget যদি সীমিত হয়, তাহলে কোন customer group / offer / campaign-এ কত টাকা দিলে সবচেয়ে বেশি incremental value পাওয়া যাবে?”

অর্থাৎ শুধু:

“কার uplift বেশি?”

না।

বরং:

“আমার হাতে ১০ লাখ টাকা আছে—এই ১০ লাখ কোথায় allocate করলে campaign-এর কারণে সবচেয়ে বেশি additional transaction/revenue/value তৈরি হবে?”

Problem

ধরো Upay-এর একটি campaign budget:

ট10,00,000

এখন ৪টা campaign আছে:

Campaign	Target	Cost	Estimated Incremental Outcome
Recharge Cashback	Recharge users	ট3L	18,000 transactions
Merchant Cashback	Active shoppers	ট2L	9,000 transactions
Bill Pay Offer	Bill pay users	ট1.5L	7,500 transactions
Send Money Offer	P2P users	ট3L	20,000 transactions

কিন্তু সব campaign-এ পুরো budget দিলে budget শেষ হয়ে যাবে।

তাই প্রশ্ন:

কোন campaign-এ কত budget দেওয়া উচিত?

আরও granular হলে:

কোন customer segment-এ কত incentive দেওয়া উচিত?

AI এখানে কী করবে?

এখানেও AI-এর কাজ শুধু budget ভাগ করা নয়।

প্রথমে আগের Uplift Model থেকে AI জানতে পারবে:

Customer/Segment



Offer দিলে কী হবে?



Offer না দিলে কী হবে?



Incremental Impact

তারপর system estimate করবে:

Incremental Impact

+

Expected Transaction Value



এরপর **Optimization algorithm** budget allocate করবে।



সবচেয়ে important distinction

Uplift Model বলে:

“এই customer-কে offer দিলে কতটা additional behavior তৈরি হতে পারে?”

Budget Optimizer বলে:

“এই information ব্যবহার করে সীমিত budget কোথায় দিলে total additional outcome maximize হবে?”

দুটো একসাথে করলে solution অনেক বেশি powerful হয়।



Complete AI + Optimization Flow



↓
Actual Outcome
↓
Feedback



Example

ধরো budget:

ট10 লাখ

তোমাদের ৪টি campaign:

Campaign A → Recharge

Campaign B → Merchant Payment

Campaign C → Bill Payment

Campaign D → Send Money

AI + optimizer estimate করল:

Campaign	Allocated Budget	Expected Incremental Transactions
Recharge	ট2.5L	16,000
Merchant	ট1.5L	8,000
Bill Pay	ট1L	5,500
Send Money	ট5L	31,000
Total	ট10L	60,500

এখানে optimizer এমন allocation খুঁজেছে যেখানে total incremental outcome বেশি হওয়ার expected value আছে।

কিন্তু এখানে একটা বড় ব্যাপার আছে

Budget allocation সবসময় linear হবে না।

ধরো:

High-frequency recharge	+12%	High	Medium	—
Dormant recharge	+45%	High	Low	—
Active merchant users	+8%	Medium	Medium	—
Frequent campaign receivers	+2%	Low	High	—

এখানে **dormant recharge users**-এর uplift বেশি হওয়ায় budget allocation বেশি হতে পারে—যদি business value এবং constraints-ও সেটা support করে।



Optimization Problem

এটাকে mathematical optimization হিসেবে formulate করা যায়।

ধরো:

Maximize:

Total Expected Incremental Value

subject to:

Total Budget $\leq$ Available Budget

Customer Eligibility = Valid

Campaign Capacity $\leq$ Maximum

Offer Cost $\leq$ Campaign Limit

Conceptually:

Maximize $\sum (\text{Incremental Value} \times \text{Allocation})$

subject to:

$\sum \text{Campaign Cost} \leq \text{Total Budget}$

এখানে **AI prediction** value estimate করবে, আর **optimization algorithm** সেই estimates ব্যবহার করে best feasible allocation বের করবে।



তাহলে AI আর Optimizer আলাদা কেন?

এটা তোমাদের presentation-এ অবশ্যই পরিষ্কার করা উচিত।

AI/ML:

Prediction

“এই segment-এ ট1 দিলে expected incremental value কত?”

Optimization:

Decision

“তাহলে মোট ট10 লাখের মধ্যে কোন segment/campaign-এ কত টাকা দেব?”

অর্থাৎ:

AI = Predict

Optimization = Decide

এটাই সবচেয়ে clean architecture।



Proposed System

আমি তোমাদের জন্য এটাকে এভাবে বানাতাম:

AI-Powered Incremental Campaign Budget Optimizer

Layer 1 — Customer Intelligence

Customer behaviour বুঝবে।

Transactions

Recharge

Merchant

Bill Pay

P2P

Recency

Frequency

Campaign history

Layer 2 — Uplift Intelligence

AI estimate করবে:

$P(\text{Response} \mid \text{Offer})$
 $P(\text{Response} \mid \text{No Offer})$

↓
Uplift

Layer 3 — Value Estimation

Uplift
×
Expected Transaction Value
-
Offer Cost

Layer 4 — Budget Optimization

Available Budget
+
Campaign Constraints
+
Expected Incremental Value
↓
Optimization Engine
↓
Budget Allocation

Layer 5 — Experiment

Actual campaign outcome measure করবে।

Layer 6 — Feedback

Actual result আবার model-এ যাবে।



Example: Real Demo Scenario

Campaign manager বলল:

“আমার budget = ৳10 লাখ। ৩টা campaign চালাতে চাই।”

System:

Campaign A — Recharge

Budget: ট0–4L
Expected incremental value curve: ...

Campaign B — Merchant

Budget: ট0–3L
Expected incremental value curve: ...

Campaign C — Send Money

Budget: ট0–6L
Expected incremental value curve: ...

তারপর optimizer:

Available Budget



AI Estimated Incremental Returns



Optimization



Recommended:

Recharge → ট2.8L
Merchant → ট1.7L
Send Money → ট5.5L

Total → ট10L

Dashboard-এ explanation:

“The recommended allocation maximizes expected incremental outcome under the given budget and campaign constraints.”



Dashboard

আমি demo-তে একটা খুব clear visualization রাখতাম:

Budget

Total Budget
ট10,00,000

Recommended Allocation

Recharge  ট2.8L
Merchant  ট1.7L

Send Money  ট5.5L

Expected Outcome

Expected Incremental Transactions
60,500

Expected Incremental Value
টX

Estimated Campaign Cost
ট10L

এবং “What if?” simulator:

Budget ট10L → ট15L করলে কী হবে?

Send Money campaign-এর budget limit 3L করলে allocation কী হবে?

এটা hackathon demo-তে খুব useful হবে।



কোন Algorithm ব্যবহার করা যায়?

প্রথমে simple রাখবে।

Prediction

- Logistic Regression
- XGBoost / LightGBM
- Uplift Model

Optimization

- Linear Programming
- Integer Linear Programming
- Mixed Integer Linear Programming
- `scipy.optimize`
- PuLP

তোমাদের use case-এ **PuLP/LP/MILP** খুব practical।



Track 04-এর অন্যগুলোর সাথে connection

এখানে আসল শক্তি হলো এটা একা থাকবে না।

Campaign Response Prediction



Uplift Modeling



Budget Optimizer

আর পাশে:

Offer Fatigue



Optimization Constraint

এবং:

Experiment Intelligence



Actual Incremental Outcome



Model Update

শেষে চাইলে:

Next Best Offer



Which offer?



Uplift



How much budget?



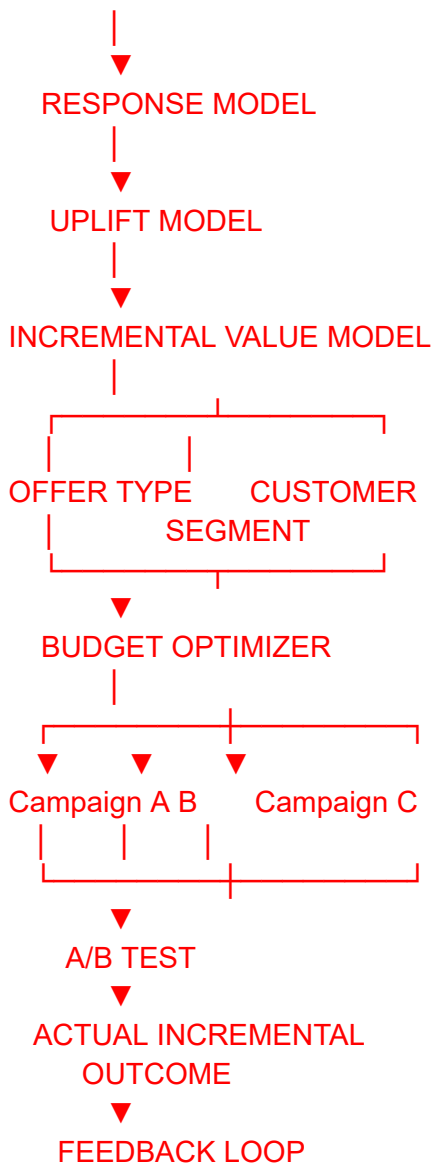
Budget Optimizer



সবচেয়ে powerful combined architecture

শেষ পর্যন্ত পুরো system হতে পারে:

CUSTOMER DATA



One-line solution

“An AI-driven budget optimization system that estimates the incremental value of different customer–offer combinations and optimally allocates a limited campaign budget to maximize incremental business outcomes.”

আর সবচেয়ে গুরুত্বপূর্ণ:

AI বলবে → কোথায় value আছে।

Optimizer বলবে → সীমিত budget-এর মধ্যে সেই value কীভাবে maximize করা যায়।

এটাই Campaign Budget Optimizer-এর proper AI solution।



Lifecycle Orchestration — Master Plan

Core Idea

Right Customer → Right Action → Right Time → Right Channel

সিস্টেমটি শুধু customer-এর বর্তমান অবস্থা বুঝবে না; বরং সে এখন lifecycle-এর কোন stage-এ আছে, এরপর কী করতে পারে, এবং তাকে কী action দিলে incremental value তৈরি হওয়ার সম্ভাবনা বেশি—সেটা AI দিয়ে নির্ধারণ করবে।

1. Problem

একজন MFS customer-এর journey একরকম নয়।

একজন:

- নতুন account খুলেছে
- প্রথম transaction করেছে
- নিয়মিত transaction করছে
- কিছুদিন ধরে activity কমিয়ে দিয়েছে
- একসময় active ছিল, এখন প্রায় inactive

কিন্তু traditional campaign system অনেক সময় সবাইকে একই ধরনের campaign পাঠায়।

ফলে:

- irrelevant campaign
 - unnecessary incentive cost
 - customer fatigue
 - low engagement
 - missed opportunities
 - organic transaction-কে campaign success হিসেবে গণনা করার সমস্যা
-

2. Our Solution

AI-Powered Customer Lifecycle Orchestration Engine

এটি প্রতিটি customer-এর জন্য continuously determine করবে:

① Where am I?

Customer বর্তমানে কোন lifecycle stage-এ?

② What will I probably do next?

তার সম্ভাব্য next behaviour কী?

③ What should we do?

কোন action নেওয়া উচিত?

④ Will the action actually change behaviour?

Offer/action দিলে incremental impact কত?

⑤ When should we do it?

কখন action দিলে response-এর সম্ভাবনা বেশি?

⑥ Through which channel?

Push / SMS / In-app ইত্যাদির মধ্যে কোন channel উপযুক্ত?

3. Lifecycle Stages

আমাদের MVP-তে ৪টি প্রধান stage থাকবে।

Stage	Customer Situation	Main Goal
Acquisition	নতুন customer	First meaningful transaction
Activation	প্রথম transaction করেছে	Usage habit তৈরি
Retention	নিয়মিত active	Engagement ধরে রাখা
Win-back	আগে active ছিল, এখন activity কম	Reactivation

4. AI-এর প্রথম কাজ — Lifecycle Detection

Customer-এর behaviour দেখে AI তার current stage estimate করবে।

Features

Transaction Frequency
Transaction Amount
Transaction Recency
Recharge Frequency
Bill Payment
Merchant Payment
Send Money
Cash-out
App Activity
Campaign Interaction
Offer Redemption
Account Age
Activity Trend
Days Since Last Transaction

Example

Customer: C1024

Account Age: 42 days
Transactions: 7
Last Transaction: 2 days ago
Average Monthly Transactions: 8
Activity Trend: Increasing

AI:

Lifecycle = Activation → Retention transition

5. 🌟 AI-এর দ্বিতীয় কাজ — Next Behaviour Prediction

শুধু stage জানলেই হবে না।

AI predict করবে:

Customer এরপর কী করতে পারে?

যেমন:

Next likely behaviour:

Recharge 72%
Merchant Payment 48%
Bill Payment 31%
Send Money 18%
Inactive 12%

এতে system বুঝতে পারবে customer-এর সম্ভাব্য next action কী।

6. Next Best Action Engine

এটাই হবে পুরো system-এর **decision layer**।

AI candidate actions তৈরি করবে এবং evaluate করবে।

Possible Actions

Offer

- Recharge cashback
- Merchant cashback
- Bill payment offer
- Transaction incentive

Money Management

- spending insight
- recharge reminder
- recurring payment reminder
- transaction pattern insight
- budget-related notification

Reminder

- bill payment
- recharge
- recurring activity

Re-engagement

- inactivity reminder
- personalized comeback message
- win-back campaign

Product Recommendation

- relevant MFS service
 - merchant payment
 - bill payment
 - other available services
-

7. 💰 Money Management — Lifecycle-এর ভিতরের Feature

এখানে খুব গুরুত্বপূর্ণ distinction থাকবে।

আমরা বলব না:

“আমরা একটি আলাদা financial management app বানিয়েছি।”

বরং:

Money Management is one of the intelligent actions within Lifecycle Orchestration.

Example

Customer নিয়মিত:

1st week → Recharge

2nd week → Recharge

3rd week → Recharge

4th week → Recharge

AI pattern detect করলো।

তারপর:

Next Best Action: Money Management → Recharge Reminder

আরেকজনের ক্ষেত্রে:

Last month spending = ট4,500

This month spending = ট6,200

AI insight:

“Your transaction spending is higher than your recent average.”

এটি **financial behaviour insight**, investment advice নয়।

8. Response Prediction

এখন system প্রতিটি possible action-এর জন্য predict করবে:

Customer এই action-এ respond করার সম্ভাবনা কত?

Example:

Action	Response Probability
Recharge Offer	78%
Merchant Offer	43%
Bill Reminder	64%
Money Insight	51%

এখানে ML model ব্যবহার করা যাবে।

MVP

Logistic Regression

Advanced

XGBoost / LightGBM

9. Uplift Modeling

এখানে আমাদের solution অনেক বেশি powerful হবে।

Response prediction বলবে:

“Customer offer পেলে respond করতে পারে।”

কিন্তু Uplift বলবে:

“Offer দেওয়ার কারণে customer-এর behaviour কতটা পরিবর্তিত হবে?”

Example:

Customer A

With Offer = 92%

Without Offer = 90%

Uplift = +2%

সে এমনিতেই transaction করত।

Customer B

With Offer = 75%

Without Offer = 25%

Uplift = +50%

এই customer-এর ক্ষেত্রে intervention-এর incremental impact অনেক বেশি।

তাই Lifecycle Engine শুধু response probability দেখবে না।

Incremental impact-ও দেখবে।

10. Expected Value

তারপর AI estimate করবে:

Expected Incremental Value

ধরা যাক:

Customer B

Uplift = 50%

Expected Transaction Value = ₹500

Expected Incremental Value

= 0.50×500

= ₹250

তারপর campaign/action-এর cost বিবেচনা করা যাবে।

11. Offer Fatigue

এটা খুব গুরুত্বপূর্ণ।

একই customer-কে বারবার offer দিলে:

Campaign 1 → Interested
Campaign 2 → Responded
Campaign 3 → Ignored
Campaign 4 → Ignored
Campaign 5 → Ignored

AI বুঝবে:

Customer may be experiencing campaign fatigue.

তখন Next Best Action হতে পারে:

No Offer

অথবা:

Money Management

অথবা:

Useful Reminder

অর্থাৎ সবসময় promotional action দেওয়া হবে না।

12. 🕒 Timing Intelligence

একই action ভুল সময়ে দিলে response কম হতে পারে।

AI historical behaviour দেখে preferred timing estimate করবে।

Example:

Customer's transaction activity

Morning → Low
Afternoon → Medium
Evening → High
8 PM → Very High

তাহলে:

Recommended Action Time → 8:00 PM

13. Channel Intelligence

একইভাবে system decide করতে পারে:

Push Notification

SMS

In-App

Example:

Push Response = 72%

SMS Response = 41%

In-App = 58%

তাহলে AI:

Recommended Channel → Push

14. Final Decision Engine

এখন সব intelligence এক জায়গায় আসবে।

Lifecycle Stage

+

Next Behaviour

+

Response Probability

+

Uplift

+

Expected Value

+

Fatigue Risk

+

Timing

+

Channel

↓

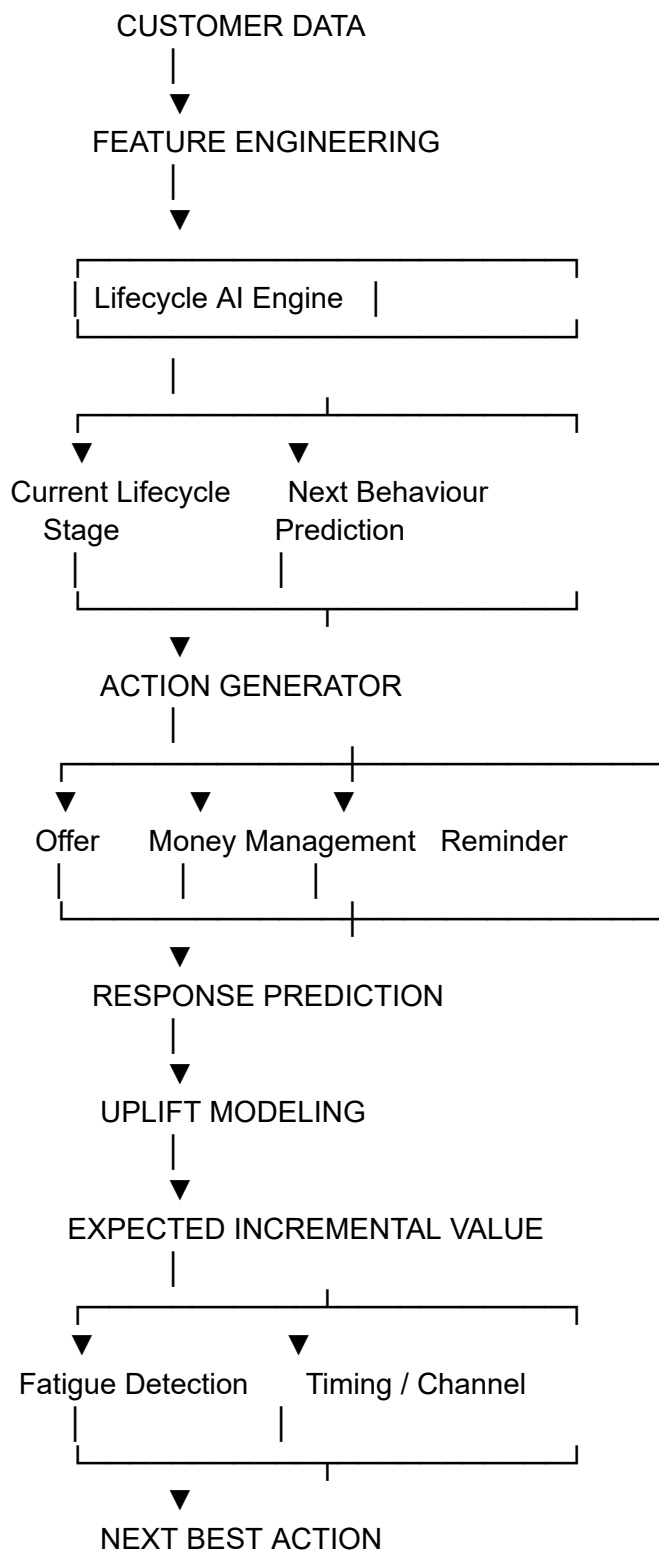
NEXT BEST ACTION

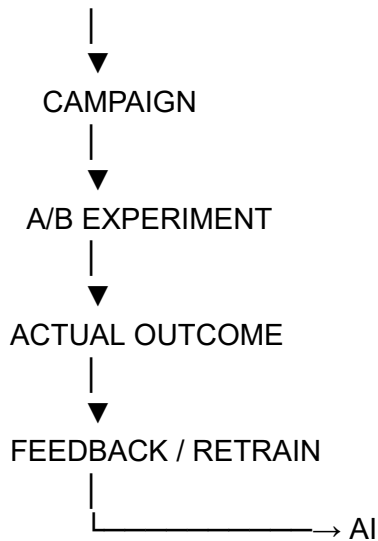
Conceptually:

Action Priority = Incremental Value + Relevance + Response – Fatigue – Cost

এটা MVP-তে simple scoring দিয়ে শুরু করা যাবে এবং পরে proper optimization model করা যাবে।

15. Complete Architecture





16. Dashboard

Hackathon demo-তে dashboard-টাই solution-টা সবচেয়ে ভালোভাবে explain করবে।

Customer 360

Customer: C1024

Lifecycle:

● Activation

Recent Activity:

Transactions: 7

Last Transaction: 2 days ago

Avg. Monthly Value: ₹3,850

Next Behaviour:

Recharge — 72%

Merchant Payment — 48%

Bill Payment — 31%

AI Recommendation

NEXT BEST ACTION

💰 Money Management

Action:

Recharge Reminder

Expected Response:

68%

Estimated Uplift:
+31%

Expected Incremental Value:
ট142

Fatigue Risk:
Low

Recommended Time:
8:00 PM

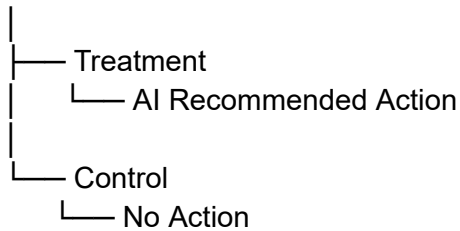
Recommended Channel:
Push Notification

17. Experiment Layer

AI recommendation সরাসরি blindly execute করবে না।

Example:

Eligible Customers



তারপর compare:

Treatment Conversion = 18.4%
Control Conversion = 11.2%

Incremental Impact = +7.2%

এই actual outcome আবার model-এ ফি্রে যাবে।

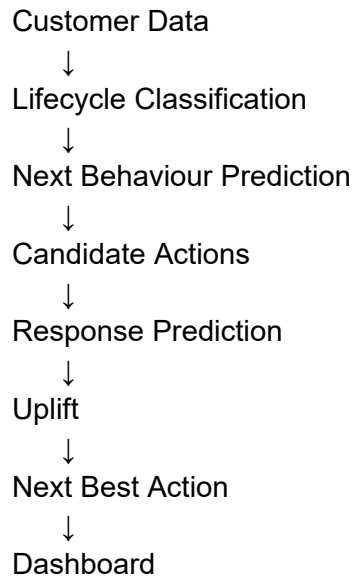
এতে system **self-improving campaign intelligence loop** তৈরি করবে।

18. MVP বনাম Advanced

● Hackathon MVP

আমরা প্রথমে সবকিছু অত্যন্ত complex করব না।

Core MVP:



Models:

- Python
- Pandas
- Scikit-learn
- Logistic Regression
- XGBoost/LightGBM
- Simple uplift T-learner
- FastAPI
- React
- SQLite/PostgreSQL

● Advanced Version

পরে add করা যাবে:

- advanced uplift models
- causal inference
- channel optimization
- timing optimization
- offer fatigue model
- explainable AI
- reinforcement learning
- campaign budget optimization
- real-time orchestration

- automated experiment intelligence
-

19. আমাদের Track 04-এর সাথে Connection

এখানেই সবচেয়ে বড় advantage।

আমরা আলাদা আলাদা feature বানাচ্ছি না।

সবগুলো একটি **Growth Intelligence ecosystem** হিসেবে কাজ করবে।

Campaign Response Prediction



Uplift Modeling



Lifecycle Orchestration



Next Best Action



Budget Optimization



Experiment Intelligence

আর **Offer Fatigue** হবে পুরো system-এর একটি safety/intelligence layer।

20. Final Product Concept

আমাদের solution-এর মূল চিন্তা হবে:

MFS customer-কে শুধু campaign পাঠানো নয়—customer-এর journey বুঝে, তার পরবর্তী সম্ভাব্য behaviour predict করে, এমন একটি action নির্বাচন করা যা সত্যিকার অর্থে incremental value তৈরি করতে পারে।

আর Money Management থাকবে সেই action ecosystem-এর একটি অংশ।

One-line Pitch

“An AI-powered lifecycle orchestration engine that understands where each customer is, predicts what they need next, and delivers the right action at the right time and through the right channel—while optimizing for incremental impact.”

এটাই আমি Lifecycle Orchestration-এর final master direction হিসেবে রাখব।

Offer Fatigue Detection — AI Hackathon Master Plan

1. Core Problem

MFS platform-এ একই customer-কে বারবার offer/campaign দেওয়া হলে তার:

- response rate কমতে পারে
- engagement কমতে পারে
- offer ignore করার প্রবণতা বাড়তে পারে
- campaign effectiveness কমতে পারে
- customer-এর ওপর unnecessary promotional pressure তৈরি হতে পারে

কিন্তু সমস্যা হলো:

সব customer একই frequency-তে fatigued হয় না।

কারণ জন্ম ৩টি offer-ই বেশি হতে পারে, আবার অন্য customer ১০টি offer-এর পরও responsive থাকতে পারে।

তাই fixed rule যথেষ্ট নয়।

আমাদের AI-এর প্রশ্ন:

“এই customer-এর historical behaviour দেখে, এখন আরেকটি promotional intervention দিলে তার response/engagement কীভাবে পরিবর্তিত হতে পারে?”

2. Solution

AI-Powered Offer Fatigue Detection & Intervention Engine

System customer-এর:

Campaign Exposure + Response History + Behaviour Change + Engagement Pattern

বিশ্লেষণ করে **personalized fatigue risk** predict করবে।

তারপর সেই prediction Lifecycle Orchestration-কে জানাবে:

**Continue Offer / Change Offer Type / Reduce Frequency / Pause Promotion
/ Use Alternative Action**

3. AI-এর Proper Role

এখানে AI-এর কাজ ৩টি মূল জায়গায়।

AI #1 — Fatigue Risk Prediction

প্রতিটি customer-এর জন্য predict করবে:

$P(\text{Fatigue} \mid \text{Customer Behaviour, Campaign History})$

উদাহরণ:

C001 → 18%

C002 → 47%

C003 → 86%

AI #2 — Behaviour Change Detection

শুধু কতগুলো offer পেয়েছে সেটা নয়।

AI customer-এর **behavioural trend** শিখবে।

যেমন:

Before Campaign Exposure

Response Rate → 68%

After Repeated Exposure

Response Rate → 24%

AI বুঝতে পারবে customer-এর engagement pattern পরিবর্তিত হচ্ছে।

AI #3 — Intervention Recommendation

Fatigue detect করার পর AI/decision engine determine করবে:

এখন customer-এর জন্য কী করা উচিত?

Possible actions:

Continue Promotion
Change Offer Category
Reduce Frequency
Pause Promotion
Money Management
Reminder
Re-engagement

এটাই আমাদের সবচেয়ে গুরুত্বপূর্ণ অংশ।

4. Data

Hackathon-এর জন্য আমরা customer-level historical campaign dataset তৈরি/ব্যবহার করতে পারি।

Customer Behaviour

customer_id
transaction_frequency
avg_transaction_value
transaction_recency
recharge_frequency
merchant_payment_frequency
bill_payment_frequency
app_activity

Campaign History

campaign_id
offer_type
offer_category
campaign_date
channel

Exposure

offers_received_7d
offers_received_30d
offers_received_90d
same_offer_count
same_category_count
days_since_last_offer

Response

clicked
opened
redeemed
transaction_after_offer
response

Behaviour Trend

response_rate_before
response_rate_recent
engagement_before
engagement_recent
transaction_frequency_before
transaction_frequency_recent

5. 🔥 সবচেয়ে গুরুত্বপূর্ণ Feature: Trend

শুধু:

```
offers_received = 10
```

এটা AI-এর জন্য খুব weak signal।

আমরা feature বানাব:

response_rate_change
engagement_change
transaction_change
campaign_response_trend

উদাহরণ:

Previous Response Rate = 64%

Recent Response Rate = 27%

Response Decline = -37%

এটা fatigue detection-এর জন্য অনেক বেশি meaningful signal।

6. 💉 AI Modeling Strategy

Hackathon-এ আমি একটি model দিয়ে সবকিছু করার চেষ্টা করব না।

বরং layered approach রাখব।

Phase 1 — Baseline

Rule-based baseline:

High exposure

+

Declining response

=

Potential fatigue

এটা থাকবে **benchmark** হিসেবে।

তারপর AI model-এর performance এর সাথে compare করব।

এতে judges-কে দেখানো যাবে:

“Simple rules vs AI model.”

7. Main AI Model

Fatigue prediction-এর জন্য:

MVP

Logistic Regression

কারণ:

- interpretable
- probability output দেয়
- দ্রুত train হয়
- explain করা সহজ

Advanced

XGBoost / LightGBM

এগুলো nonlinear relationship ধরতে পারবে।

যেমন:

Offer Frequency

+

Customer Type

+

Response Trend

+

Offer Similarity

+

Time Since Last Offer

মিলে fatigue risk তৈরি হচ্ছে কিনা।

8. Model Output

Model output হবে:

Customer ID

Fatigue Probability

Risk Level

Top Contributing Factors

Example:

Customer: C2048

Fatigue Probability: 0.84

Risk: HIGH

Key Factors:

- 9 offers in last 30 days
- Same category repeated 6 times
- Response rate dropped 41%
- Last 3 campaigns ignored

এখানে **Explainable AI** ব্যবহার করলে demo অনেক stronger হবে।

9. Explainable AI

শুধু:

“Fatigue = 84%”

বললে judge জিজ্ঞেস করতে পারে:

Why?

তাই system explanation দেবে।

Example:

Why is fatigue high?

- + High campaign exposure
- + Repeated recharge offers
- + Declining response rate
- + Recent campaign ignores
- Long gap since previous campaign

Advanced implementation:

SHAP

এতে model-এর prediction-এর জন্য কোন feature কতটা contribute করেছে সেটা দেখানো যাবে।

10. Customer-Specific Fatigue

এখানে আমাদের AI-এর সবচেয়ে logical use দেখা যাবে।

ধরা যাক:

Customer A

Offers/month = 8

Response rate = 72%

AI:

Fatigue Risk = 12%

Customer B

Offers/month = 5

Response rate = 18%

Response trend = sharply declining

AI:

Fatigue Risk = 81%

অর্থাৎ:

Same exposure ≠ Same fatigue

এটাই AI-এর value।

11. 🎁 Category-Level Fatigue

আরও intelligent করার জন্য fatigue শুধু overall হবে না।

AI estimate করতে পারে:

Recharge Offer Fatigue
Merchant Offer Fatigue
Bill Payment Offer Fatigue
Generic Promotional Fatigue

Example:

Category	Fatigue
Recharge	88%
Merchant	21%
Bill Payment	35%

তখন পুরো customer-কে promotional system থেকে বাদ দেওয়ার দরকার নেই।

AI বলতে পারে:

Pause Recharge Promotions; Merchant engagement remains viable.

12. 🔗 Lifecycle Orchestration Integration

এখানে Offer Fatigue-এর আসল value।

Lifecycle Engine-এর কাছে candidate actions থাকবে:

A → Recharge Offer
B → Merchant Offer
C → Money Management
D → Reminder
E → No Promotional Action

তারপর বিভিন্ন AI signal আসবে:

Lifecycle Stage

+

Response Probability

+

Uplift

+

Fatigue Risk

+

Expected Value

তারপর final action I

13. Example — Full Decision

Customer:

Lifecycle = Retention

Recharge Response = 76%

Recharge Uplift = 8%

Recharge Fatigue = 88%

Merchant Response = 54%

Merchant Uplift = 31%

Merchant Fatigue = 18%

Money Management Relevance = High

Fatigue = Low

Traditional system:

Recharge offer পাঠাও — response 76%।

আমাদের AI system:

Recharge-এর response high হলেও incremental impact low এবং fatigue high।

তাই:

Next Best Action

Merchant Engagement

অথবা context অনুযায়ী:

Money Management Insight

এটাই আমাদের solution-এর intelligence।

14. Cooldown Prediction

Fatigue high হলে শুধু “stop” বলা যথেষ্ট নয়।

AI estimate করতে পারে:

কতদিন promotional intervention না দিলে customer-এর responsiveness recover হতে পারে?

Historical data থেকে:

3 days → recovery probability 21%

7 days → 48%

14 days → 72%

তারপর:

Recommended cooldown: 7 days

এটি advanced feature হিসেবে রাখা যাবে।

Hackathon MVP-তে fixed cooldown baseline রাখা যায়, কিন্তু advanced demo-তে ML-based recovery prediction দেখানো যাবে।

15. Counterfactual / Uplift Integration

এখানে আমাদের overall Track 04 solution-এর সবচেয়ে powerful connection।

Fatigue prediction বলছে:

“Repeated offer-এর কারণে fatigue risk high।”

Uplift model বলছে:

“এই offer দিলেও incremental impact মাত্র +5%।”

তখন decision:

Do not send promotional offer.

কিন্তু অন্য customer:

Fatigue = Low

Uplift = +42%

Expected Value = High

তখন:

Send intervention.

অর্থাৎ:

Fatigue $\neq$ Decision

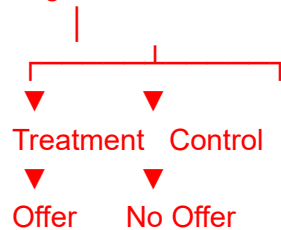
Fatigue হলো একটি decision signal।

16. Experiment Intelligence

আমরা model-এর prediction blindভাবে বিশ্বাস করব না।

Customer-দের experiment করা হবে।

Eligible Customers



তারপর measure:

Response Rate

Transaction Rate

Revenue

Engagement

Unsubscribe / Ignore

এতে আমরা validate করতে পারব:

AI যাদের fatigue-prone বলেছে, তাদের intervention দিলে সত্যিই response/engagement খারাপ হচ্ছে কিনা।

17. Model Evaluation

Hackathon-এ শুধু model বানিয়ে accuracy দেখানো উচিত না।

Classification Metrics

- ROC-AUC
- PR-AUC
- Precision
- Recall
- F1

বিশেষ করে fatigue class imbalance থাকলে **PR-AUC** useful।

Business Metrics

আরও গুরুত্বপূর্ণ:

Campaign Response Improvement

Unnecessary Offer Reduction

Incremental Conversion

Customer Engagement Preservation

Campaign Cost Saved

18. Hackathon Dashboard

Dashboard-এর main screen এমন হতে পারে:

OFFER FATIGUE INTELLIGENCE	
Customers Analyzed	125,000
High Fatigue Risk	18,420
Medium Risk	31,280
Low Risk	75,300
Avg. Campaign Exposure	6.2
Avg. Response Decline	18.4%

তারপর:

Customer-level view

Customer: C2048

Fatigue Risk

● 84%

Exposure

9 offers / 30 days

Response Trend

64% → 23%

Category Fatigue

Recharge ● 88%

Merchant ● 21%

Bill Pay ● 39%

AI Explanation

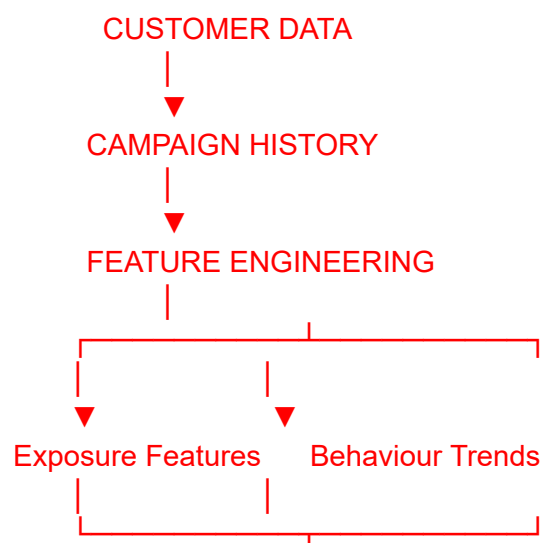
- High repeated exposure
- Response declining
- Same offer category repeated

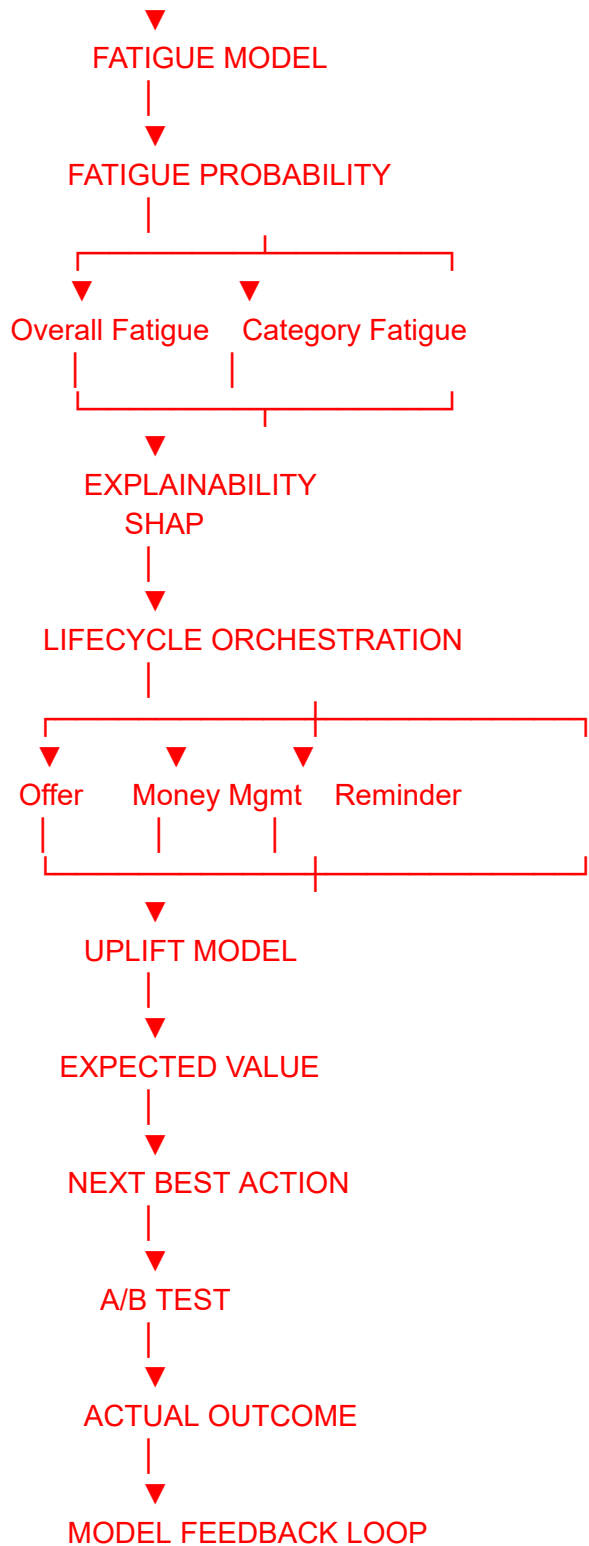
AI Recommendation

Pause Recharge Offers

Consider Merchant Engagement

19. 🏗️ Complete Technical Architecture





20. 🛠 Hackathon Implementation Stack

আমি এখানে unnecessary technology ঢেকাব না।

Backend

Python
Pandas
Scikit-learn
XGBoost
SHAP
FastAPI

Database

SQLite

MVP-এর জন্য যথেষ্ট।

Frontend

React
Recharts

Model Pipeline

Data
↓
Feature Engineering
↓
Train/Test Split
↓
Baseline Rule Model
↓
ML Fatigue Model
↓
Probability
↓
SHAP Explanation
↓
Decision Engine
↓
Dashboard

21. 🏆 Hackathon-এর জন্য আমাদের AI Story

Presentation-এ আমরা “আমরা AI ব্যবহার করেছি” বলব না।

বরং story হবে:

Step 1

Rules can count exposure.

“Customer 10টা offer পেয়েছে।”

কিন্তু rules জানে না customer-এর behaviour কেন পরিবর্তন হচ্ছে।

Step 2

ML learns individual fatigue patterns.

“এই customer-এর response history দেখে fatigue probability 84%।”

Step 3

Explainable AI tells us why.

“High exposure + repeated category + declining response।”

Step 4

Uplift tells us whether intervention is still worthwhile.

“Offer দিলে incremental impact মাত্র 8%।”

Step 5

Lifecycle Orchestration chooses the action.

“Recharge offer বন্ধ রেখে merchant engagement / money-management action।”

Step 6

Experiment validates the decision.

Treatment vs Control → actual incremental outcome.

এভাবে AI-এর ব্যবহার **logical, measurable এবং business-driven** হবে।

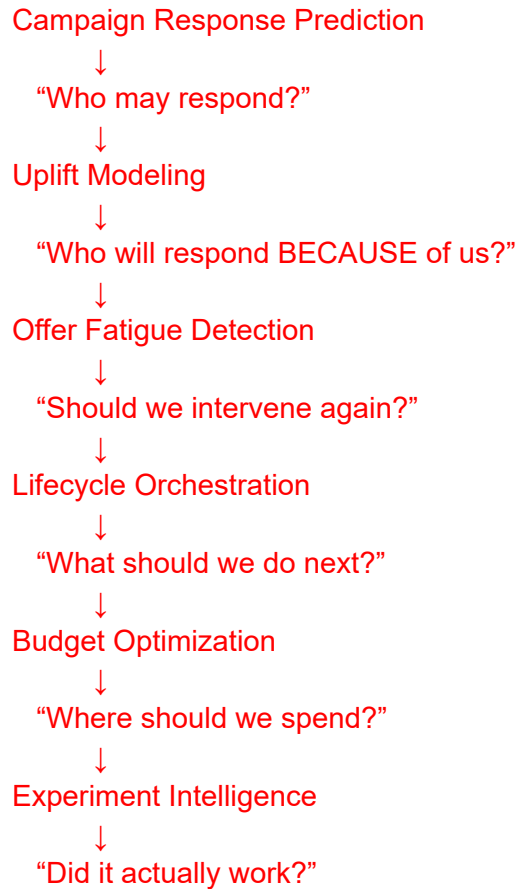
22. Final Concept

আমাদের Offer Fatigue module-এর final definition:

“An AI system that learns individual customer engagement patterns from campaign exposure, response history, and behavioural trends to predict

personalized offer-fatigue risk, explain its drivers, and recommend whether to continue, change, reduce, or pause promotional interventions.”

আর পুরো Track 04 solution-এ এর position:



এটাই আমাদের AI hackathon-এর জন্য সবচেয়ে logical framing: প্রতিটি AI component আলাদা আলাদা buzzword নয়—একটির prediction আরেকটির decision-কে feed করবে, এবং শেষ পর্যন্ত **incremental growth**-এর দিকে যাবে।

7. Experiment Intelligence — Final Master Plan

1. Core Idea

Experiment Intelligence হলো এমন একটি AI system যা MFS-এর পূর্বের campaign/experiment-এর ফলাফল থেকে শিখে:

কোন customer segment-এর জন্য, কোন offer, timing, channel বা message-এর কোন combination পরবর্তীতে test করা উচিত—তা intelligently recommend করবে।

অর্থাৎ:

Experiment → Learn → Discover Pattern → Generate Hypothesis →
Recommend Next Experiment → Test → Learn Again

2. 🚨 Problem Statement

MFS platform প্রতিনিয়ত বিভিন্ন campaign চালায়:

- cashback
- discount
- recharge offer
- merchant offer
- bill payment incentive
- SMS vs Push
- different timing
- different message

কিন্তু সাধারণ A/B testing system সাধারণত শুধু বলে:

“Variant B performed better than Variant A.”

এতে একটি experiment-এর ফলাফল পাওয়া যায়, কিন্তু পরবর্তী experiment কী হওয়া উচিত, সেটা automatically বের হয় না।

আরও গুরুত্বপূর্ণ:

একটি campaign overall ভালো করলেও সেটা সব customer-এর জন্য ভালো নাও হতে পারে।

আমাদের system তাই জানতে চাইবে:

“What did we learn from previous experiments, where did it work, where did it fail, and what should we test next?”

3. 💡 Proposed Solution

AI-Powered Experiment Intelligence Engine

System historical experiments-এর data collect করে:

Step 1

Experiment outcome analyse করবে।

Step 2

Customer segment অনুযায়ী pattern খুঁজবে।

Step 3

Successful এবং unsuccessful intervention-এর কারণ/relationship identify করবে।

Step 4

Untested বা under-tested combinations খুঁজবে।

Step 5

Potential experiment generate করবে।

Step 6

প্রতিটি experiment-এর expected impact, cost এবং learning value estimate করবে।

Step 7

একটি explainable recommendation দেবে।

4. Experiment Data

প্রতিটি experiment-এর জন্য আমরা রাখব:

Experiment Metadata

experiment_id
campaign_id
experiment_type
start_date
end_date

Treatment

control
variant_A
variant_B

Customer Context

lifecycle_stage
customer_segment
transaction_frequency
avg_transaction_value
activity_level

Intervention

offer_type
cashback_percentage
message_type
channel
timing
frequency

Outcome

response
conversion
transaction
transaction_value
revenue
campaign_cost

Incremental Metrics

uplift
incremental_transactions
incremental_revenue
ROI

5. 🧠 AI-এর প্রথম কাজ — Experiment Analysis

AI historical experiment analyse করবে।

Example:

Experiment #24

Control → 8.1%
10% Cashback → 12.7%
15% Cashback → 14.2%

System শুধু বলবে না:

15% performed better.

বরং দেখবে:

Which segment?

Which lifecycle stage?

Which timing?

Which channel?

Which customer behaviour?

6. Segment-Level Intelligence

ধরা যাক:

Segment	Control	10%	15%
Acquisition	8%	11%	12%
Activation	10%	16%	18%
Retention	14%	16%	17%
Win-back	5%	14%	21%

AI pattern detect করবে:

Higher cashback appears particularly promising for Win-back customers.

কিন্তু এটাকে শুধু correlation হিসেবে treat না করে experiment results/statistical evidence দিয়ে validate করা হবে।

7. AI-এর মূল কাজ — Pattern Discovery

AI historical experiments থেকে relationship শিখবে।

Possible features:

Lifecycle Stage

Customer Segment

Offer Type

Offer Value

Channel
Time
Day
Campaign Frequency
Previous Response
Fatigue Risk
Transaction Behaviour

তারপর model estimate করবে:

Given this context, how promising is this experiment configuration?

এখানে supervised ML ব্যবহার করা যেতে পারে।

MVP

- Logistic Regression
- Random Forest / XGBoost

Advanced

- Bayesian optimization
 - contextual bandit
-

8. Experiment Candidate Generation

এখন system নিজে possible experiments তৈরি করবে।

ধরা যাক historical data:

Offer:
10%, 15%

Channel:
Push, SMS

Timing:
Morning, Evening

Segment:
Activation, Retention, Win-back

System সম্ভাব্য combinations তৈরি করবে:

15% + Win-back + Push + Evening
10% + Activation + SMS + Morning

15% + Retention + Push + Evening

এরপর যেগুলো ইতিমধ্যে পর্যাাপ্তভাবে test হয়েছে সেগুলো বাদ দেবে।

9. 🧠 Next Experiment Scoring

প্রতিটি candidate experiment-এর জন্য system কয়েকটি dimension evaluate করবে।

① Expected Incremental Impact

এই experiment successful হলে expected uplift কত?

② Expected Business Value

সম্ভাব্য incremental transaction/revenue কত?

③ Experiment Cost

Test চালাতে কত cost হবে?

④ Learning Value

এই experiment থেকে নতুন কতটা information পাওয়া যাবে?

⑤ Confidence

Historical evidence কতটা strong?

⑥ Customer Risk

High fatigue বা negative engagement-এর risk আছে কিনা?

10. 🎯 Learning Value — আমাদের AI-এর বিশেষ অংশ

একটি experiment শুধু business outcome-এর জন্য নয়, learning-এর জন্যও valuable হতে পারে।

ধরা যাক:

Experiment A

Expected uplift: High

Cost: High

Learning: Low

কারণ একই ধরনের experiment অনেকবার হয়েছে।

Experiment B

Expected uplift: Medium

Cost: Low

Learning: High

কারণ এই segment + offer combination আগে test করা হয়নি।

তখন Experiment B valuable হতে পারে কারণ এটি system-এর uncertainty কমাবে।

11. AI Recommended Experiment

শেষে system একটি actionable recommendation তৈরি করবে।

Example

Recommended Experiment

Target: Win-back customers

Control: No promotional intervention

Variant A: 15% Recharge Cashback

Variant B: Personalized Money Management Message

Timing: Evening

Channel: Push

Why?

- ✓ Win-back segment historically responds to incentives
- ✓ Recharge campaigns show diminishing response after repetition
- ✓ Current fatigue risk is moderate
- ✓ Money-management intervention is under-tested
- ✓ This experiment can distinguish promotional vs non-promotional re-engagement

এখানেই AI-এর reasoning visibly বোঝা যাবে।

12. Statistical Validation

AI recommendation-এর আগে/পরে statistical testing থাকবে।

কারণ:

Higher conversion $\neq$ necessarily better experiment

আমরা দেখব:

- sample size
- confidence interval
- statistical significance
- treatment/control difference
- minimum detectable effect

Example

Control = 10.1%

Variant = 10.6%

Difference = +0.5%

AI বলবে না:

“Variant is better.”

বরং:

Observed improvement is small and may not yet provide sufficient evidence for a meaningful effect.

এতে system scientifically stronger হবে।

13. Uplift Modeling Integration

এটা আমাদের পুরো project-এর core intelligence-এর সাথে connect হবে।

Normal experiment:

Treatment Conversion

vs

Control Conversion

থেকে আমরা calculate করব:

Incremental Impact

তারপর Experiment Intelligence সেই historical uplift data ব্যবহার করবে।

Example:

Win-back + 15% cashback

→ historical uplift: +12%

Retention + 15% cashback

→ historical uplift: +3%

AI পরবর্তী experiment-এর candidate scoring-এ এগুলো ব্যবহার করবে।

14. Offer Fatigue Integration

Experiment Intelligence previous campaign fatigue-ও consider করবে।

Example:

Recharge cashback

→ historically successful

BUT

Same customers

→ repeated exposure

→ response declining

→ fatigue increasing

তখন AI recommendation হতে পারে:

Test a non-promotional intervention instead of increasing cashback.

এখানে:

Fatigue Detection → Experiment Intelligence

একটি important feedback relationship তৈরি করবে।

15. Lifecycle Integration

Experiment recommendation lifecycle-aware হবে।

Acquisition

Welcome message
vs
Welcome cashback

Activation

Second transaction cashback
vs
Usage reminder

Retention

Personalized offer
vs
Generic offer

Win-back

Cashback
vs
Money Management engagement

অর্থাৎ একই experiment সবার জন্য recommend করা হবে না।

16. 💰 Budget Optimization Integration

প্রতিটি experiment-এর estimated cost থাকবে।

System দেখাতে পারে:

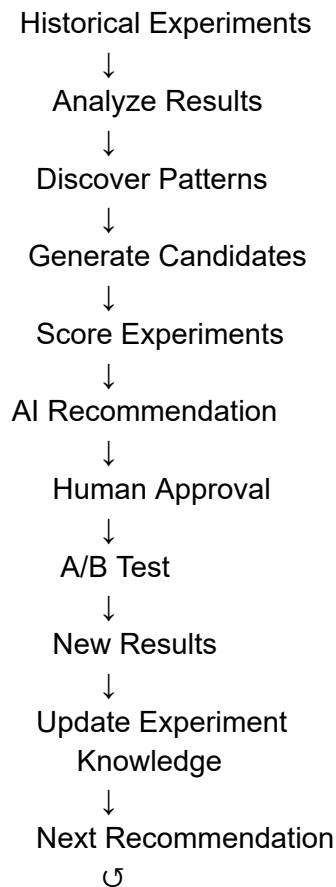
Experiment	Expected Uplift	Cost	Learning Value
A	High	High	Medium
B	Medium	Low	High
C	High	Medium	High

এতে campaign team বুঝতে পারবে:

Limited experimentation budget কোথায় ব্যবহার করলে business value এবং learning—দুটোই পাওয়া যেতে পারে।

17. Continuous Learning Loop

এটিই পুরো feature-এর heart:



এভাবে system static dashboard না হয়ে **learning system** হবে।

18. Hackathon Dashboard

Experiment Intelligence Overview

EXPERIMENT INTELLIGENCE			
Experiments Run	48		
Significant Results	31		
Avg Incremental Uplift	7.4%		
Recommendations Generated	12		

Historical Insight

Experiment #32

Segment:

Win-back

Best observed intervention:

15% Cashback

Incremental Uplift:

+7.9%

Confidence:

95%

AI Recommendation

 NEXT EXPERIMENT	
Segment: Win-back	
Control	
→ No promotional intervention	
Variant A	
→ 15% Cashback	
Variant B	
→ Money Management Message	
Timing: Evening	
Channel: Push	
Expected Impact: High	
Learning Value: High	

19. AI Explainability

Recommendation-এর নিচে:

Why did AI recommend this?

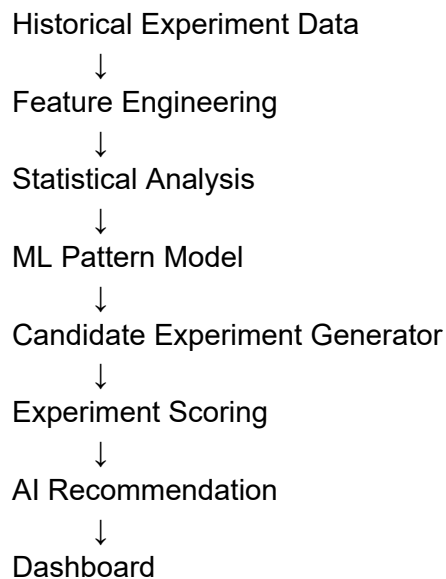
1. Win-back customers showed strong uplift

- from incentive-based campaigns.
2. Recharge campaigns show declining response after repeated exposure.
 3. Current fatigue level is moderate.
 4. Money-management intervention has limited historical testing.
 5. The experiment can distinguish promotional vs non-promotional re-engagement.
- এটা judges-এর জন্য অত্যন্ত useful হবে।
-

20. Hackathon MVP

আমরা প্রথম version-এ খুব বেশি complex করব না।

MVP Architecture



Tech Stack

Python
Pandas
Scikit-learn
XGBoost
SciPy / Statsmodels
FastAPI

21. Advanced Version

যদি time/data থাকে:

Contextual Bandits

System dynamically learns:

Which action works best for which customer context?

Bayesian Optimization

Efficiently search করবে:

Best offer value + timing + segment combination.

Causal Inference

Correlation-এর বদলে:

Did the intervention actually cause the outcome?

Automated Experiment Design

AI automatically suggest করবে:

- target population
- treatment
- control
- sample size
- experiment duration
- primary metric

22. গুরুত্বপূর্ণ Design Principle

আমাদের AI যেন **নিজে** business decision silently execute না করে।

Recommended flow:

AI Recommendation
↓
Human Review
↓
Experiment Approval
↓
Controlled Test
↓
Measurement

এতে system practical এবং trustworthy হবে।

23. 🏆 Final AI Story

Hackathon presentation-এ Experiment Intelligence-কে এভাবে explain করা সবচেয়ে শক্তিশালী হবে:

“Traditional experimentation tells us what happened. Our Experiment Intelligence layer learns why different interventions worked for different customer contexts and uses those learnings to recommend what should be tested next.”

আর পুরো logic:

PAST EXPERIMENTS
↓
WHAT WORKED?
↓
FOR WHOM?
↓
UNDER WHAT CONDITIONS?
↓
WHAT HAS NOT BEEN TESTED?
↓
WHAT IS WORTH TESTING NEXT?
↓
RUN EXPERIMENT
↓
MEASURE INCREMENTAL IMPACT
↓
LEARN
↓
REPEAT

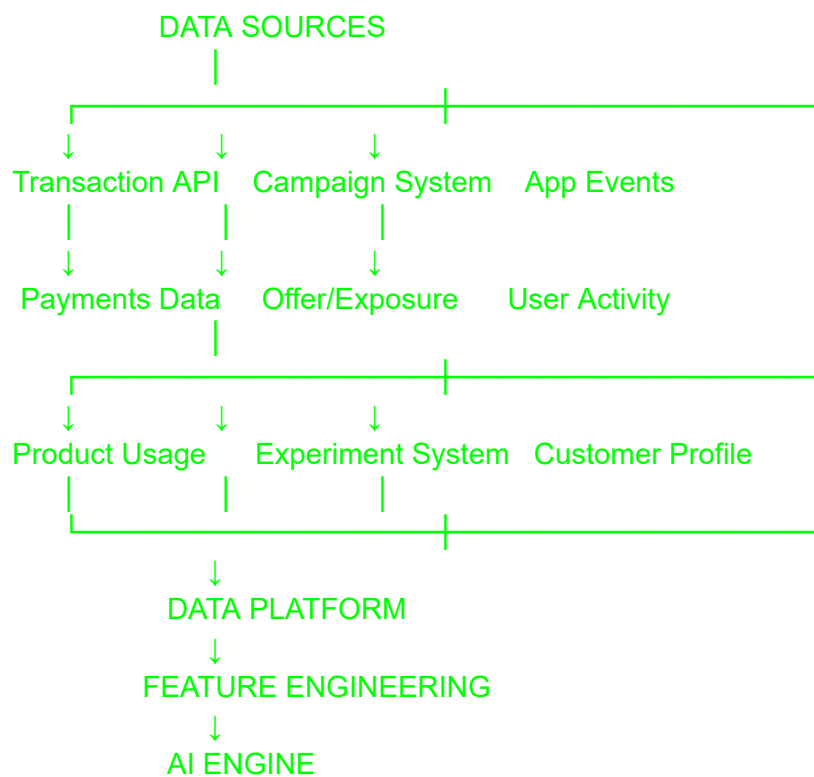
Final one-line definition

Experiment Intelligence is an AI-driven continuous learning engine that analyzes historical campaign experiments, discovers segment-level patterns, evaluates potential hypotheses, and recommends the next experiment based on expected incremental impact, business value, customer fatigue, cost, and learning value.

এভাবে **Experiment Intelligence** আমাদের project-এর “learning brain” হবে—যেটা আগের campaign থেকে শিখে পরের campaign intelligence আরও উন্নত করবে।

1. Data কোথা থেকে আসবে?

Real MFS-এর ক্ষেত্রে data এক জায়গা থেকে আসবে না। বিভিন্ন source থাকবে।



তোমাদের prototype-এ

যেহেতু real MFS-এর production database পাওয়া যাবে না, demo-তে এই sourceগুলো **simulated APIs / synthetic event generator / CSV ingestion** দিয়ে দেখানো যাবে।

কিন্তু UI-তে source-গুলো real-world architecture-এর মতোই দেখাবে।

2. Exactly কী data collect করবে?

আমি এটাকে ৪টা data stream-এ ভাগ করব।

Data Source	কী data আসবে	কোন feature কাজে লাগবে
Transaction	type, amount, timestamp	Response, Uplift, Lifecycle
Product Usage	recharge/merchant/bill-pay usage	Lifecycle, NBO
Campaign	campaign, offer, channel	Response, Fatigue
Campaign Response	open/click/redeem/transaction	Response, Uplift
Experiment	control/variant/outcome	Uplift, Experiment Intelligence
Engagement	login, session, activity	Lifecycle, Timing
Customer Profile	segment, account age	Segmentation
Consent/Preference	consent, channel preference	Eligibility & safety

3. Transaction data

User যখন transaction করবে, backend event তৈরি করবে।

Example:

```
{
  "customer_id": "C1024",
  "transaction_id": "TX83921",
  "type": "merchant_payment",
  "amount": 850,
  "timestamp": "2026-10-02T18:32:00"
}
```

Authority চাইলে Raw Data Explorer-এ এটা দেখতে পারবে।

তারপর feature engine বানাবে:

```
transactions_30d = 12
avg_transaction_30d = 640
merchant_payment_30d = 7
days_since_last_transaction = 1
```

AI raw transaction-এর বদলে primarily এই derived behavioural features ব্যবহার করবে।

4. Campaign data

যখন authority/customer-এর জন্য campaign চালাবে:

Campaign ID: CMP-102
Offer: Merchant Cashback
Cashback: 10%
Channel: Push
Start: 6 PM
End: 10 PM

প্রত্যেক exposure-এর event থাকবে:

C1024
↓
Campaign CMP-102
↓
Delivered
↓
Opened
↓
Clicked
↓
Redeemed
↓
Transaction

এতে system বুঝতে পারবে:

কাকে কী দেওয়া হয়েছিল এবং তার পরে কী হয়েছিল।

5. Experiment data

এটা খুব important।

ধরো authority test করছে:

Experiment: Merchant Cashback Test

Control → No offer
Variant A → 10% cashback
Variant B → 15% cashback

System record করবে:

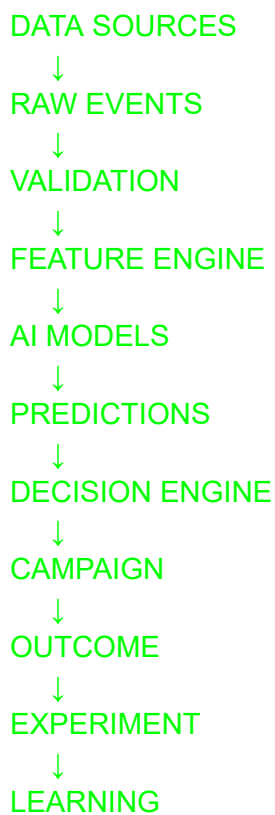
Customer → Variant
Customer → Response
Customer → Transaction
Customer → Transaction Value

এটাই পরে uplift model-এর জন্য সবচেয়ে valuable data।

6. Authority Dashboard

এখানে আমি একটা বড় “Data Lineage” section রাখব।

Authority dashboard-এ এমন flow থাকবে:



প্রতিটা box clickable হবে।

7. Authority Raw Data Explorer

Authority চাইলে দেখতে পারবে:

Transaction Events

Time	Customer	Type	Amount	Source
18:32	C1024	Merchant	ট850	Transaction API
19:05	C2048	Recharge	ট500	Transaction API

Campaign Events

Time	Customer	Campaign	Channel	Event
18:00	C1024	CMP102	Push	Delivered
18:04	C1024	CMP102	Push	Opened
18:08	C1024	CMP102	Push	Redeemed

Authority তখন বুঝবে:

“এই data আসছে কোথা থেকে?”

8. কিন্তু শুধু raw data দেখালে হবে না

সবচেয়ে powerful অংশ হবে **Data** → **AI Trace**।

Authority একজন customer select করবে:

Customer C1024

RAW DATA

12 transactions
7 merchant payments
5 recharge
6 campaign exposures
2 campaign responses

↓

FEATURE ENGINE

Transactions 30D = 12
Avg Txn Value = ট640
Merchant Frequency = 7

Recharge Frequency = 5
Days Since Last Txn = 1
Campaign Response Rate = 33%
Fatigue Risk Features = Medium

↓

AI MODELS

Response Model
 $P(\text{Response}) = 71\%$

Uplift Model
With Offer = 78%
Without Offer = 42%
Uplift = +36%

Fatigue Model
Risk = 18%

Lifecycle Model
Stage = Activation

↓

DECISION ENGINE

Recommended Action:
Merchant Cashback

Expected Incremental Value:
৳X

Recommended Channel:
Push

Recommended Time:
7:00 PM

এটা judge-কে দেখালে খুব পরিষ্কার হবে AI কোথায় কাজ করছে।

9. Authority-এর নিজের AI Assistant

এটা তোমাদের system-এর আরেকটা বড় feature হতে পারে।

Authority-এর জন্য আলাদা:

Growth Intelligence Assistant

Authority natural language-এ প্রশ্ন করবে।

যেমন:

“এই সপ্তাহে campaign performance কেমন?”

Assistant উত্তর:

This week:

Campaigns: 18

Customers reached: 1.24M

Average response: 8.4%

Estimated incremental transactions: 41,200

High-fatigue customers suppressed: 86,400

Authority বলতে পারবে:

“Why did the system recommend merchant cashback for C1024?”

Assistant:

C1024 is currently in the Activation stage. The customer has recently shown higher merchant-payment activity, while recharge offers have a higher fatigue risk. The uplift model estimates a larger incremental response from merchant cashback.

আবার বলতে পারবে:

“Generate a campaign performance report for September.”

AI report generate করবে:

September Growth Intelligence Report

1. Campaign Overview
2. Customer Reach
3. Response Rate
4. Incremental Impact
5. Campaign Cost
6. Estimated Value
7. Fatigue Analysis
8. Lifecycle Distribution
9. Experiment Results

10. Recommendations

এটা PDF/Excel/management report হিসেবেও generate করা যাবে।

10. সবচেয়ে important — Assistant যেন শুধু chatbot না হয়

এখানে একটা architecture rule রাখব:



মানে AI নিজে data বানিয়ে answer করবে না।

Authority যদি জিজ্ঞেস করে:

“Last 30 days-এ কত customer high fatigue?”

Assistant actual database/model output query করবে।

তারপর:

“83,412 customers currently have high fatigue risk.”

এবং চাইলে:

View Data →

click করলে সেই customer list দেখা যাবে।

11. AI Assistant-এর প্রতিটি answer-এর সাথে “Why?” থাকবে

এটা তোমাদের জন্য খুব valuable।

ধরো assistant বলল:

Merchant campaign recommended for 32,400 customers.

তার নিচে:

Why?

- ✓ High merchant engagement
- ✓ Positive historical uplift
- ✓ Low merchant-offer fatigue
- ✓ Suitable lifecycle stage
- ✓ Expected incremental value

আর:

Data Sources

Transaction Events
Campaign History
Experiment Results
Customer Behaviour Features

অর্থাৎ authority শুধু answer পাবে না—

answer-এর পিছনের evidence-ও দেখতে পারবে।

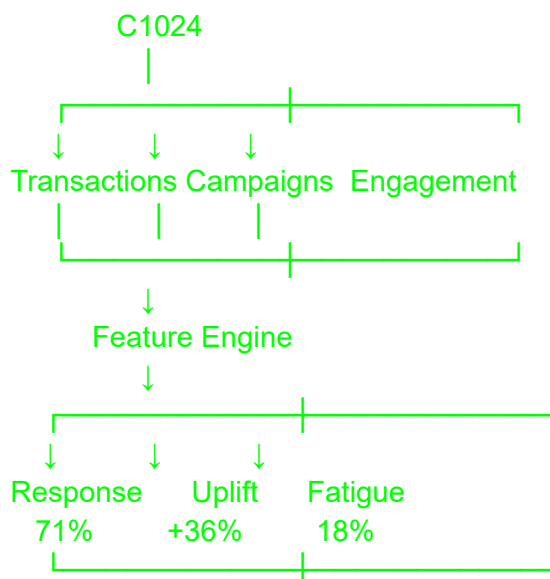
12. পুরো Authority Panel আমি এভাবে সাজাব

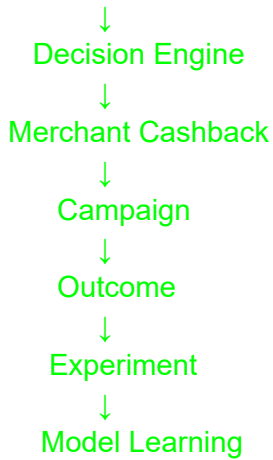
GROWTH INTELLIGENCE CONTROL CENTER			
Overview			
└ Growth KPIs			
└ Incremental Value			
└ Active Campaigns			
└ AI Recommendations			
DATA			
└ Data Sources			
└ Raw Events			
└ Data Quality			
└ Data Lineage			



13. Data Lineage — এটা অবশ্যই রাখবে

Authority যদি **C1024** select করে:



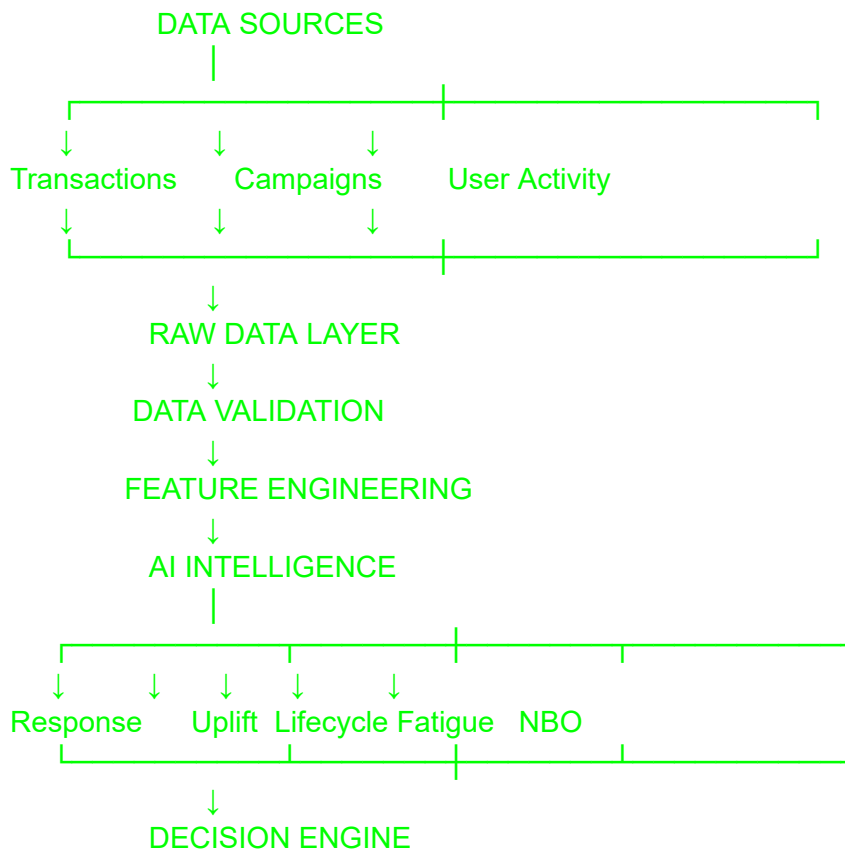


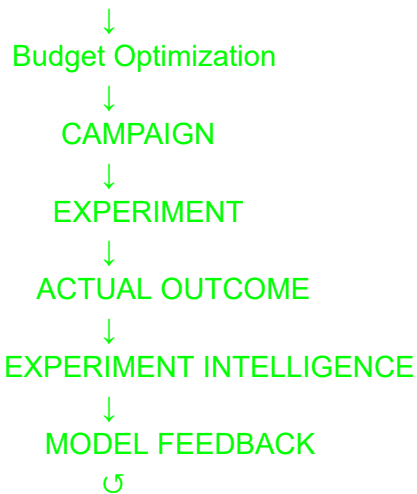
প্রতিটা node click করলে details খুলবে।

এটা basically তোমাদের system-এর **AI Explainability + Data Transparency layer**।

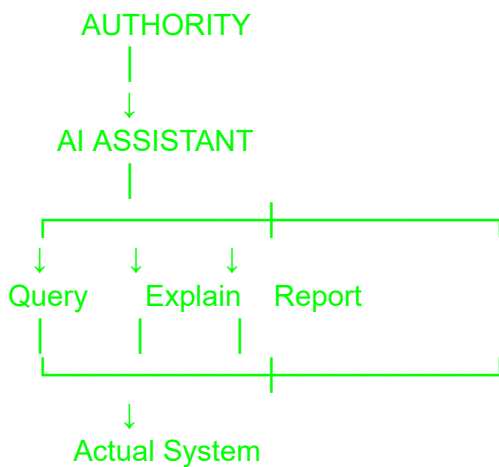
14. তাহলে 7টা feature আসলে আলাদা আলাদা না

Final architecture হবে:





আর পাশে:



Judge-এর সামনে সবচেয়ে concise answer হবে:

“We collect consented behavioural events from transaction systems, campaign systems, product usage and experimentation systems. These raw events are stored with source metadata, validated, and transformed into behavioural features. Different AI models then predict campaign response, incremental uplift, lifecycle stage, fatigue and next behaviour. A decision layer converts those predictions into actions such as next-best-offer and budget allocation. Every recommendation is traceable back to its underlying features and source events. Our authority panel exposes this entire data lineage, while an authority-side AI assistant can query the actual system, explain why a decision was made, and generate evidence-based reports.”

DATASET

H- Growth Intelligence — Developer Master Implementation Plan

1. Objective

Build a hackathon-ready AI-powered Growth & Campaign Intelligence Platform where we do not depend on a real MFS production database.

Instead, the platform will support a Synthetic User Dataset / JSON ingestion system.

Each user can have a structured JSON profile containing:

- Customer profile
- Balance / financial snapshot
- Transaction history
- Product/service usage
- Campaign exposure
- Campaign responses
- Experiment history
- Preferences and consent
- Behavioural information

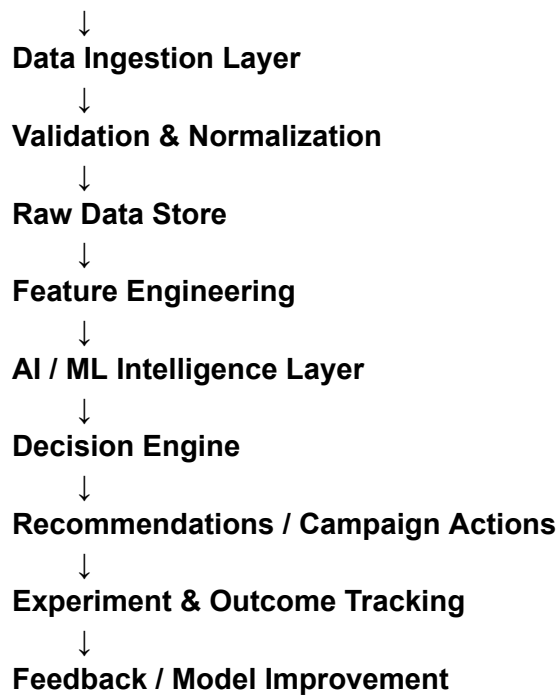
The system will ingest this JSON, convert raw data into ML features, run the AI models, generate recommendations, and expose the complete reasoning/data lineage through the Authority Dashboard.

The architecture must be designed so that later the JSON data source can be replaced by real MFS APIs/database without rebuilding the AI layer.

2. Core Architecture

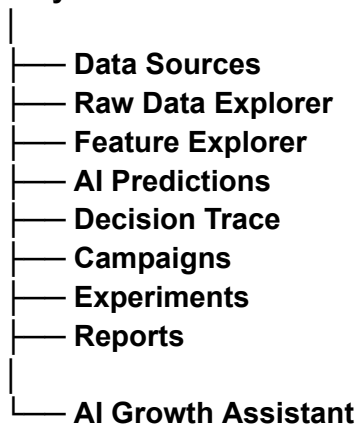
Implement the following pipeline:

JSON User Dataset



Authority interface:

Authority Dashboard



3. JSON Dataset System

3.1 User JSON

Create a standard schema for each user.

Example:

```
{  
  "customer": {  
    "customer_id": "C1024",
```

```
"segment": "active",
"account_age_days": 420,
"region": "Dhaka"
},

"financial_snapshot": {
  "balance": 4250,
  "currency": "BDT"
},

"transactions": [
  {
    "transaction_id": "TX001",
    "type": "merchant_payment",
    "amount": 850,
    "timestamp": "2026-09-28T18:30:00"
  },
  {
    "transaction_id": "TX002",
    "type": "recharge",
    "amount": 500,
    "timestamp": "2026-09-30T21:10:00"
  }
],

"service_usage": {
  "recharge_count_30d": 5,
  "merchant_payment_count_30d": 7,
  "bill_payment_count_30d": 2,
  "send_money_count_30d": 3
},

"campaign_history": [
  {
    "campaign_id": "CMP101",
    "offer_type": "recharge_cashback",
    "channel": "push",
    "status": "redeemed",
    "timestamp": "2026-09-20T18:00:00"
  }
],

"experiments": [
  {
    "experiment_id": "EXP01",
    "variant": "B",
    "offer": "merchant_cashback_10",
    "response": true
  }
]
```

```
}  
],  
  
"preferences": {  
  "preferred_channel": "push",  
  "recharge_offer_allowed": true,  
  "merchant_offer_allowed": true,  
  "marketing_consent": true  
}  
}
```

The schema should be extensible.

Do NOT hardcode the AI around this exact JSON structure. Create an ingestion/normalization layer so the AI consumes a standardized internal schema.

4. Data Ingestion

Authority should have:

Load User Dataset

Options:

1. Upload JSON
2. Select existing synthetic user
3. Run batch dataset

Flow:

Upload JSON
↓
Schema Validation
↓
Data Preview
↓
Import
↓
Generate Features
↓
Run AI Analysis

Display:

✓ Customer Profile

- ✓ Financial Snapshot
- ✓ 42 Transactions
- ✓ 8 Campaign Events
- ✓ 3 Experiment Records
- ✓ Preferences

If validation fails, show exact errors.

Example:

Invalid field:

transactions[4].amount

Expected:

number

Received:

string

5. Raw Data Layer

Store the ingested data separately from derived AI features.

Minimum logical entities:

customers

transactions

service_usage

campaigns

campaign_exposures

campaign_responses

experiments

experiment_results

preferences

raw_events

Every raw record should preserve:

- source
- ingestion timestamp
- customer_id
- event type
- original event ID where available

Example:

source = synthetic_json

source_file = C1024.json
ingested_at = ...

This is required for data lineage.

6. Feature Engineering Layer

Do not send the entire transaction history directly into every model.

Create a Feature Engine.

Example features:

Transaction

transactions_7d
transactions_30d
transactions_90d
avg_transaction_30d
total_transaction_value_30d
days_since_last_transaction

Service

recharge_count_30d
merchant_payment_count_30d
bill_payment_count_30d
send_money_count_30d
cashout_count_30d

Campaign

offers_received_7d
offers_received_30d
campaign_response_rate_30d
campaign_redemption_rate_30d
same_category_offer_count
days_since_last_offer

Engagement

activity_trend
preferred_hour
preferred_channel

Experiment

treatment_group

control_group
historical_uplift
experiment_response_rate

The Feature Engine must expose these features to the Authority Dashboard.

7. AI Intelligence Layer

Implement the following AI modules.

7.1 Campaign Response Prediction

Input:

Customer behavioural features

+

Campaign features

Output:

response_probability

Example:

C1024

Merchant Cashback

Response Probability = 71%

Start with a simple interpretable baseline such as Logistic Regression.

Then support a stronger tree-based model such as XGBoost/LightGBM if appropriate.

8. Uplift Modeling

This is one of the core AI components.

The model must estimate:

$P(\text{response} \mid \text{offer})$

$P(\text{response} \mid \text{no offer})$

Then:

uplift =
 $P(\text{response} \mid \text{offer})$
-
 $P(\text{response} \mid \text{no offer})$

Example:

With Offer = 78%
Without Offer = 42%

Estimated Uplift = +36%

For MVP, implement a T-Learner / two-model approach.

Do not only predict response.

The UI must clearly distinguish:

Response Probability
vs
Incremental Uplift

9. Next Best Offer

Generate candidate offers:

Recharge Cashback
Merchant Cashback
Bill Payment Offer
Send Money Offer
No Promotional Action

Before AI ranking:

Apply eligibility rules

Check:

- marketing consent
- user preferences
- offer eligibility
- campaign constraints

Then AI scores candidates.

Example:

Merchant Cashback
Response: 71%
Uplift: +36%
Fatigue: 18%
Expected Value: High

Recharge Cashback
Response: 74%
Uplift: +7%
Fatigue: 81%
Expected Value: Low

Recommendation:

Next Best Offer:
Merchant Cashback

The decision must be explainable.

10. Lifecycle Orchestration

Predict the current lifecycle stage:

Acquisition
Activation
Retention
Win-back

Then predict likely next behaviour.

Example:

Current Stage:
Activation

Predicted Next Behaviour:

Recharge	72%
Merchant Payment	48%
Bill Payment	31%
Send Money	18%

Generate candidate next actions:

Offer
Reminder

Money Management
Product Recommendation
Re-engagement
No Promotional Action

Then rank actions using:

Response
+
Uplift
+
Lifecycle Relevance
-
Fatigue
-
Cost

11. Money Management

Money Management is NOT a separate domain.

It is an action category inside Lifecycle Orchestration.

Examples:

Recharge reminder
Recurring payment reminder
Spending insight
Transaction pattern insight
Budget-related notification

The system should treat these as possible next actions alongside promotional actions.

12. Offer Fatigue Detection

Create an AI model that predicts personalized fatigue risk.

Useful features:

offers_received_7d
offers_received_30d
same_category_offer_count

days_since_last_offer
response_rate_trend
redemption_rate_trend
engagement_trend

Output:

fatigue_probability
fatigue_level

Example:

Recharge Fatigue
88% — HIGH

Merchant Fatigue
18% — LOW

Do not use only a fixed rule such as:

offers > 5 = fatigue

Rules can be a baseline, but the final system should use a learned model.

13. Budget Optimizer

Separate prediction from optimization.

AI predicts:

Expected response
Expected uplift
Expected incremental value

Optimization engine decides:

How to allocate limited budget

Inputs:

campaign
segment
expected uplift
expected value
campaign cost
available budget
campaign capacity

fatigue constraints

Example:

Total Budget = ₺10,00,000

Campaign A = ₺3,20,000

Campaign B = ₺4,50,000

Campaign C = ₺2,30,000

Use a simple optimization library such as PuLP/scipy.optimize for MVP.

14. Experiment Intelligence

Track:

experiment_id

customer_id

variant

offer

channel

timing

response

transaction

transaction_value

cost

The system should:

- 1. Analyze previous experiments**
- 2. Calculate treatment/control outcomes**
- 3. Calculate incremental impact**
- 4. Discover segment-level patterns**
- 5. Generate candidate next experiments**
- 6. Rank candidates based on expected value + learning value + cost + fatigue**

Do not claim a variant is better from a tiny raw difference without appropriate statistical validation.

15. Decision Engine

Create one central Decision Engine.

Inputs:

Response Score

Uplift Score

Lifecycle Stage

Fatigue Risk

Expected Value

Offer Cost

User Preference

Campaign Constraints

Output:

Recommended Action

Recommended Offer

Recommended Channel

Recommended Timing

Example:

Customer: C1024

Action:

Merchant Cashback

Channel:

Push

Timing:

7:00 PM

Expected Response:

71%

Expected Uplift:

+36%

Fatigue Risk:

18%

16. Data Lineage / Explainability

This is mandatory.

Every AI recommendation must be traceable:

Raw Data



Features



Model



Prediction



Decision

For example:

Customer C1024



Transactions

Campaign History

Experiment History



merchant_frequency_30d = 7

campaign_response_rate = 33%

days_since_last_txn = 1



Uplift Model



Merchant Uplift = +36%



Decision Engine



Merchant Cashback Recommended

Authority must be able to click each stage.

17. Authority Dashboard

Create the following sections.

Overview

Show:

Total Customers

Active Campaigns

Campaign Reach

Response Rate

Estimated Incremental Transactions
Estimated Incremental Value
High Fatigue Customers
Active Experiments

Data Sources

Show:

Synthetic JSON
Transaction Events
Campaign Events
Experiment Data
User Activity

For each source show:

- records
- last ingestion
- status
- source type

Raw Data Explorer

Authority can select:

Customer
Event Type
Date
Campaign

and inspect raw records.

Feature Explorer

Show:

transactions_30d
avg_transaction_30d
merchant_frequency
campaign_response_rate
fatigue_features

AI Predictions

Show:

Response
Uplift
Lifecycle
Fatigue
Next Behaviour

Decision Trace

Show:

Why was this action recommended?
Which features were used?
Which model generated the prediction?
What constraints were applied?

18. Authority AI Assistant

Build a separate AI Assistant for authority.

The assistant must NOT invent data.

Architecture:

Authority
↓
AI Assistant
↓
Query / Analytics Layer
↓
Actual Database + AI Outputs
↓
Answer

Supported questions:

"Why was merchant cashback recommended for C1024?"

"How many customers have high fatigue risk?"

"What was our best performing campaign last month?"

"Show me customers with high uplift."

"Why was this customer excluded from the campaign?"

"How much budget was allocated to merchant campaigns?"

"Generate September campaign performance report."

The assistant should retrieve actual system data before answering.

19. Report Generation

Authority can ask:

Generate September Growth Report

Generate:

**Executive Summary
Campaign Overview
Customer Reach
Response Rate
Incremental Impact
Campaign Cost
Expected Value
Fatigue Analysis
Lifecycle Distribution
Experiment Results
Budget Allocation
AI Insights**

Allow export to PDF/CSV where practical.

20. User-Level AI View

Authority should be able to select:

Customer C1024

and see:

PROFILE

**Balance
Account Age
Transaction Summary**

Service Usage
Campaign History
Experiment History

AI INTELLIGENCE

Lifecycle:
Activation

Response:
71%

Uplift:
+36%

Fatigue:
18%

Next Behaviour:
Recharge

Next Best Action:
Merchant Cashback

Recommended Channel:
Push

Recommended Time:
7 PM

Then show:

Why?

- ✓ Strong merchant activity
 - ✓ Positive historical response
 - ✓ High estimated incremental impact
 - ✓ Low merchant fatigue
 - ✓ User has marketing consent
-

21. Dataset Modes

Implement two modes.

Single User Mode

Load JSON

- **Analyze User**
- **Show User Intelligence**

Useful for demoing the full reasoning.

Batch Mode

Load 1,000 / 10,000 synthetic users

- **Feature Engineering**
- **Run Models**
- **Generate Predictions**
- **Campaign Targeting**
- **Dashboard Analytics**

This demonstrates that the system can scale beyond one user.

22. Synthetic Data Generator

Create a generator for realistic MFS-like behaviour.

Do NOT generate completely random data.

The generator should create behavioural patterns such as:

Active users

Dormant users

New users

Win-back users

High-value users

Offer-sensitive users

Offer-fatigued users

Merchant-heavy users

Recharge-heavy users

It should also generate campaign treatment/control data so uplift modeling and experiments can actually run.

Example hidden behavioural patterns:

Win-back customers

- may respond to relevant reactivation offers

Repeated promotional exposure

- may increase fatigue

Highly active customers

→ may transact without an offer

Relevant offers

→ may create higher incremental impact

The ML models should learn from observable features rather than receiving these hidden labels directly.

23. Important AI Architecture Rule

Do NOT build:

JSON → LLM → Recommendation

Instead build:

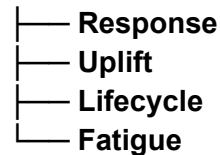
JSON



Feature Engineering



ML Models



Optimization / Decision Engine



Recommendation



LLM Assistant

The ML models make predictions.

The optimization/decision layer makes business decisions.

The LLM is mainly used for:

- Natural-language interaction
- Explanation
- Report generation
- Data querying
- Authority assistance

This makes the AI architecture much more defensible in the hackathon.

24. Production-Ready Data Source Abstraction

Create a common interface:

```
DataSource
├── JSONDataSource
├── SyntheticDataSource
└── FutureMFSAPIDataSource
```

For the hackathon:

JSONDataSource

is active.

Later:

Real MFS API

can replace it without changing:

Feature Engine
AI Models
Decision Engine
Dashboard
AI Assistant

This is important for explaining scalability to judges.

25. Privacy / Security

Do not use:

- PIN
- password
- NID
- raw authentication credentials
- unnecessary phone numbers
- unnecessary exact location

Use synthetic/anonymized customer IDs.

Balance may exist for dashboard/context, but avoid using unnecessary sensitive raw financial information as model features.

Store only data needed for the intelligence functions.

26. Final End-to-End Demo Flow

The demo should follow this exact story:

1. Authority opens Dashboard

2. Authority selects:
"Load User Dataset"

3. Uploads:
C1024.json

4. System validates JSON

5. System shows:
✓ 42 transactions
✓ 8 campaign events
✓ 3 experiments
✓ preferences

6. Click:
"Run AI Analysis"

7. Feature Engine runs

8. AI models run

9. Dashboard shows:

Lifecycle:
Activation

Response:
71%

Uplift:
+36%

Fatigue:
18%

10. Decision Engine recommends:

Merchant Cashback

11. Authority clicks:
"Why?"

12. System shows:

Raw Data
↓
Features
↓
AI Prediction
↓
Decision

13. Authority opens AI Assistant

14. Asks:
"Why did you recommend this offer?"

15. Assistant explains using actual model outputs.

16. Authority asks:
"Generate campaign report."

17. System generates report.

18. Authority can inspect the underlying data at every stage.

27. Core Product Principle

The entire product should communicate one idea:

"Every AI decision is traceable from source data to feature to prediction to business action to actual outcome."

The system should therefore not look like a generic AI chatbot.

It should look like:

28. MVP Priority

Build in this order:

Phase 1 — Data Foundation

- JSON schema
- JSON upload
- validation
- raw data storage
- synthetic dataset generator

Phase 2 — Feature Engine

- transaction features
- campaign features
- engagement features
- lifecycle features
- fatigue features

Phase 3 — Core AI

- Response Prediction
- Uplift Modeling
- Lifecycle Prediction
- Fatigue Prediction

Phase 4 — Decision Layer

- Next Best Offer
- Next Best Action
- Budget Optimization

Phase 5 — Experiment Intelligence

- experiment tracking
- treatment/control analysis
- experiment recommendations

Phase 6 — Authority Dashboard

- raw data explorer
- feature explorer
- AI predictions
- decision trace
- campaign dashboard

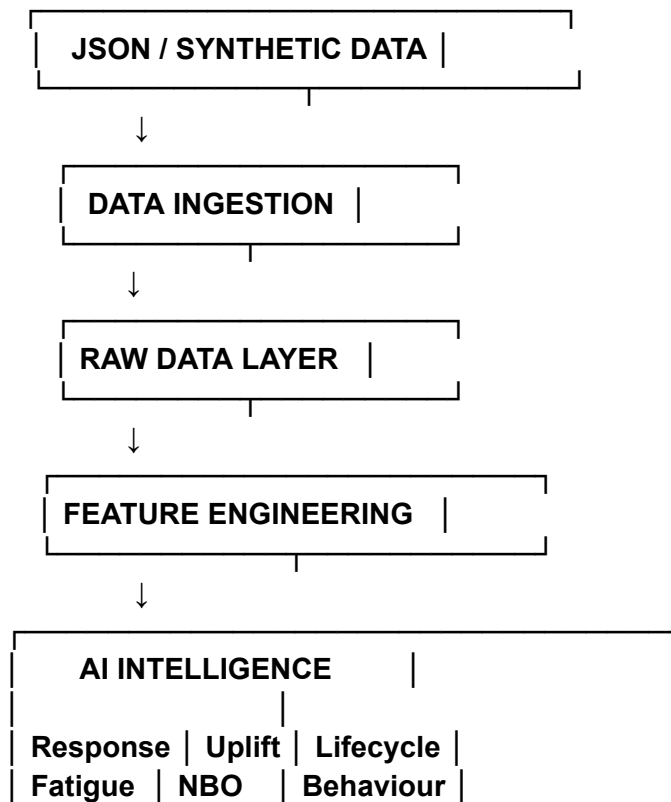
Phase 7 — Authority AI Assistant

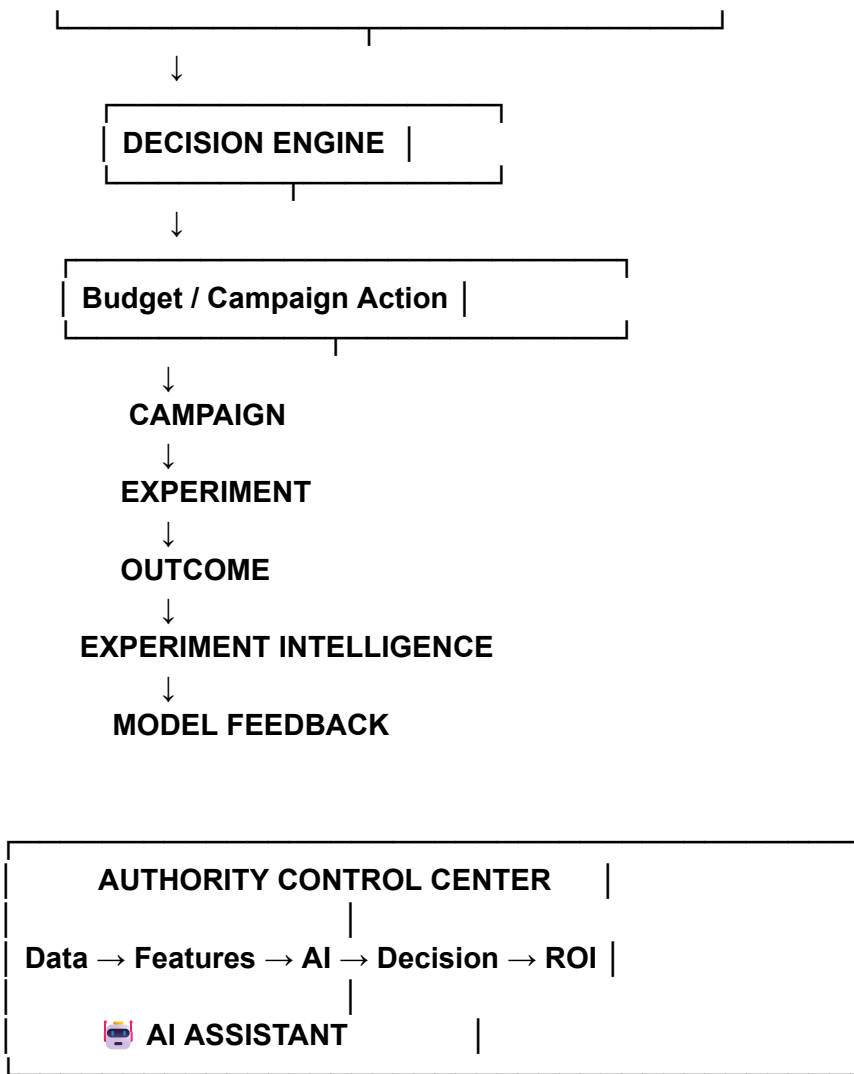
- system Q&A
- explain decisions
- customer analysis
- report generation

Phase 8 — Polish

- data lineage visualization
- explainability
- batch processing
- demo scenarios
- export/reporting
- error handling

Final Architecture





Developer-এর জন্য মূল requirement: UI যতই সুন্দর হোক, demo-তে অবশ্যই এই traceability visibly কাজ করতে হবে:

JSON → Raw Data → Features → AI Prediction → Decision → Explanation → Outcome

এটাই পুরো system-এর backbone।

